LIDAR GUIDED ROVER FOR LANDMINE DETECTION

Final Year Project

By

Muhammad Khizer Sheraz – 403629

Muhammad Umer Haroon – 410709

Muhammad Abu Bakr Hashim – 411421

In Partial Fulfillment

Of the Requirements for the degree

Bachelor of Electrical Engineering (BEE)

School of Electrical Engineering and Computer Science

National University of Sciences and Technology

Islamabad, Pakistan

(2026)

DECLARATION

We hereby declare that this Final Year Project report entitled **"LiDAR Guided Rover for Landmine Detection"** submitted to the **School of Electrical Engineering and Computer Sciences (SEECS), National University of Sciences and Technology (NUST), Islamabad**, is a record of our original work carried out under the supervision of **Dr. Neelma Naz** and co-supervision of **Dr. Latif Anjum**.

We further declare that no part of this report has been plagiarized and that all sources of information, technical references, software frameworks, and supporting material used during the course of this project have been properly acknowledged through appropriate citations and references. This report has not been submitted, either in full or in part, for the award of any other degree or academic qualification.

Team Members

Muhammad Khizer Sheraz _____

Muhammad Umer Haroon _____

Muhammad Abu Bakr Hashim _____

Supervisor

Dr. Neelma Naz _____

Co-Advisor

Dr. Latif Anjum _____

Date: _____

Place: National University of Sciences and Technology (NUST, H-12), Islamabad

DEDICATION

This project is dedicated to our parents whose love, sacrifices, prayers, and the continuous encouragement has been our greatest source of motivation in the course of our studies. The unrelenting support they have given us has helped us to achieve the goals we have set out with confidence and persistence.

We also dedicate this work to our teachers and mentors whose guidance, discipline, and commitment to knowledge have played a critical role in our technical understanding and professional development. Their messages have given us the framework on which this project was to be constructed.

Lastly, we would like to dedicate this project to the humanitarian workers, engineers, security personnel and deminers who work in mine affected areas in the different parts of the world. The boldness and efforts in their defense of human lives motivated them to come up with this project. We hope that further progress in the autonomous robotics and intelligent sensing systems will further minimize human exposure to danger and help to make landmines survey and detection activities safer and more effective.

ACKNOWLEDGEMENTS

It is with great pleasure that we wish to show our great appreciation to our respected supervisor, **Dr. Neelma Naz**, due to her great guidance, the constant support and thoughtful feedback during the course of our Final Year Project. Her support, technical understanding, and academic guidance were a key factor in us being able to figure out what direction we were heading to with this work and how to improve the quality of this work at every step.

We also owe our co-advisor, **Dr. Latif Anjum**, a huge debt of gratitude to give him the technical advice, constructive recommendations and support in the design and implementation phases of the project. His feedback allowed us to narrow down on key engineering choices to do with system architecture, robotics workflow, and simulation-based development.

We have recognized the assistance of the **Department of Electrical Engineering** and the **School of Electrical Engineering and Computer Sciences (SEECS), NUST** in availing us the academic environment, facilities, and institutional platform we needed to accomplish this project successfully. The fact that we got the chance to work in such an environment helped us a great deal to learn and develop our project.

We are also grateful to our classmates and friends to have discussed, given suggestions, and encouraged us all through the project timeline. Their comments and ethical encouragement helped the development process to become less complicated and much more effective.

And lastly, we want to convey our deepest gratitude to our families who have been so patient, praying, motivating and still believing in us and our capabilities. Their support was always the strength to motivate us through the hardest stages of development, testing, debugging, and report writing.

ABSTRACT

Landmines continue to pose a post-conflict threat, and they still threaten lives and restrict land use decades after the wars. Conventional methods of detection involve human operators and thus makes it a slow, costly and risky process. This brings out the necessity of autonomous or semi-autonomous robotic systems capable of functioning in hazardous settings and reducing human exposure to risk.

This project describes the design and development of a **LiDAR Guided Rover** to detect **landmines**, which was based on the **Articubot One** mobile robot architecture, and modified to suit a humanitarian demining use case. The system was built on **ROS 2 Humble** and **Gazebo Classic** with a heavy emphasis on autonomous navigation, simulation-based validation, modular robot modelling, and ROS-based software integration. The rover has been designed as a differential-drive mobile robot with LiDAR-based perception to detect obstacles, map the environment and navigate, whilst landmine detection was modeled through a detection mechanism implemented into the overall ROS workflow.

The project includes major concepts and subsystems of robotics such as **URDF/Xacro based robot modelling, ROS nodes, topics, TF transforms, Gazebo-ROS integration, SLAM, Navigation Stack (Nav2)** and an **Arduino based low level motor control interface**. The rover can sense the surrounding using LiDAR scans, construct or utilize a map of the environment, position itself, and move on its own, avoiding immobile obstacles. The conceptual design of a mine detection layer is such that suspected locations of mines can be identified and published into the ROS ecosystem, allowing visualization of events and testing of the system in simulation.

A key engineering lesson of this project was the design change between a **four wheel drive** design which had initially been considered to a **two wheel differential-drive** design with **caster support**. The four wheel system also created problems concerning the **wheel slip, odometry drift**, and erratic motion dynamics, which had an adverse effect on localization and the reliability of the odometry. The two wheel plus caster design on the other hand provided more predictable kinematics, lower odometric inconsistency, and more compatibility with standard ROS differential-drive controllers. This design enhancement greatly enhanced system stability and simulation performance.

The finished system shows that it is possible to create a low-cost autonomous rover with the help of open source robotics tools and simulation first methodology to use the system in landmine survey applications. Although the current implementation is based on simulation and system integration, it offers a high technical quality in the future improvement of the large-area inspection of the minefield using numerous robots.

Keywords: *LiDAR, ROS 2 Humble, Gazebo Classic, autonomous rover, landmine detection, Articubot One, Nav2, SLAM, differential drive robot, obstacle avoidance, TF, URDF, Arduino motor interface*

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABBREVIATIONS

ABBREVIATION	FULL FORM
ROS 2	Robot Operating System 2
UGV	Unmanned Ground Vehicle
LiDAR	Light Detection and Ranging
URDF	Unified Robot Description Format
Xacro	XML Macro
TF	Transform Framework
SLAM	Simultaneous Localization and Mapping
Nav2	Navigation 2 Stack
RViz	ROS Visualization Tool
GUI	Graphical User Interface
Gazebo	Robot Simulation Environment
MCU	Microcontroller Unit
PWM	Pulse Width Modulation
IMU	Inertial Measurement Unit
CAD	Computer-Aided Design
USB	Universal Serial Bus
RPLIDAR	Rotating Platform LiDAR
AMCL	Adaptive Monte Carlo Localization
ODOM	Odometry
CMD_VEL	Command Velocity

DDS	Data Distribution Service
PID	Proportional Integral Derivative
RPM	Revolutions Per Minute
ROS-Gazebo Integration	Communication interface between ROS 2 and Gazebo simulation
Diff Drive	Differential Drive
Costmap	Cost Map used by navigation stack for obstacle representation
Topic	ROS communication channel for message passing
Node	Executable process in ROS
Launch File	ROS file used to start multiple nodes and configurations

Chapter 1: Introduction

1.1 Introduction

Landmines have remained to be one of the most threatening unseen dangers of the post-conflict regions [1]. Landmines can remain buried over many years and keep injuring people long after a war is over. They render agricultural land unusable, retard the rehabilitation process, threaten transport routes and instill a permanent sense of fear in communities living within the affected areas. Removing the mines is not always the only problem, but, first, locating where the danger might be and not subject human beings to unnecessary risk.

It is here that robotics will be of significance. An unmanned ground rover may be employed as a first-response survey tool rather than deploying a person into a possibly dangerous location. This type of rover will be able to navigate through the area, sense any dangers, have a map of the surroundings, and aid the process of the detection of the suspected mine locations. Although the robot may not be accomplishing the entire mine disposal task, it can still help to reduce the risk by performing the risky early-stage mine inspection work.

This project aims at developing a **LiDAR Guided Rover** to detect landmines using a modern robotics software workflow based on **ROS 2 Humble** and **Gazebo Classic**. The system is modeled on the design philosophy of **Articubot One** [12] but is adjusted to a mine-detection themed use case. The rover was simulated as a mobile differential-drive robot that can move autonomously, navigate around obstacles, simulate detection behavior within a structured ROS environment.

Another significant aspect of the project was not only to construct a robot model, but to construct a whole robotics workflow around it. That involved robot description using **URDF/Xacro**, integrating controllers, simulating LiDAR sensors, communicating via ROS topics, publishing transforms, simulating in Gazebo Classic, visualizing in RViz, and aiding navigation using the ROS 2 ecosystem. In this sense, the project turned out to be much more than just a mere mobile robot. It turned into a full-scale exercise in integrating the robotic system.

The other significant aspect of the work was the optimization of the physical design of the rover. The project was not initiated with the last two-wheel differential-drive arrangement. A previous effort had been to adopt a **four-wheel rover layout**, but this layout presented issues related to motion, particularly **wheel slip**, and **odometry shift**, that affected the stability of the localization and the reliability of navigation. As development continued, it became apparent that a **two-wheel drive with caster support** was the superior choice since it was more closely matched with the kinematic assumptions of standard differential-drive control and produced more predictable odometric behaviour.

The end product is a humanitarian-driven project with a solid foundation in robotics engineering. It shows how an autonomous rover can be designed and tested in simulation to be able to be utilised in landmine-related applications, but also it would be a strong learning platform to model mobile robots, to design a system using ROS, and integrate a navigation-stack.

1.2 Background and Motivation

The drive behind this project is based on a humanitarian approach, as well as a technical approach. In the humanitarian aspect, landmines are not only military artifacts that have remained in the hands of the military, but they are also still elements that affect the daily lives of ordinary people. A mine buried may turn arable land into a mine hazard, or render a pathway unusable, or it may leave an entire community reliant on slow and expensive clearance operations. Under these circumstances, any technology that minimizes human direct contact with hazard is desirable.

Technically speaking, robotics has now reached its maturity, so undertaking projects such as this one can be done using the available tools and open source systems. Previously, constructing an autonomous rover meant special robotics platforms and very tailored software. A student team can now use **ROS 2, Gazebo, RViz**, and structured robot description files, to create a fairly advanced mobile robot workflow. This accessibility is one of the major factors that led to the possibility of this project to be implemented in the current form.

Another reason that inspired the project was the need to stop the isolated code or simulation exercises and instead, develop a system that feels complete. Instead of learning about SLAM as a standalone project, learning about navigation as a standalone project, and learning about robot modelling as a standalone project, this project has put all those pieces together into one coherent application. The rover was required to live as a model, act like a robot, publish the correct transforms, accept motion commands, sense its surroundings, and operate within a simulated environment of a mission. It is typically in this type of integration where the actual engineering problem is.

The concept of working with an Articubot-style workflow, in particular, was particularly attractive as it offered a sensible starting point and yet, still allowed for customization. Its general design was also common and suited to the differential-drive navigation, but the context of the mission in this project was different. The robot was required to be packaged as a rover that would be used to survey hazardous terrain and scout mines. That necessitated a more intense focus on environmental consciousness, rover stability, and the reasoning of how a robot should act when a potential detection event transpires.

The four-wheeled to two-wheeled design change also turned out to be a significant cause of motivation. Theoretically, a four-wheel platform will be more secure and rough. Practically, however, the simulation outcomes and motion behavior indicated that the additional wheels did not necessarily enhance the performance. The four-wheel design spawned undesired complexity since the odometry and motion model of the rover had to be consistent with the ROS differential-drive control assumptions. The decision to go back to a 2-wheel and caster system was not a regression, but a design choice to make the system more reliable and the robot act more truthfully within the ROS navigation pipeline.

1.3 Problem Context

The main issue that this project is going to solve is how to develop a mobile robotic platform that would be able to move independently in an unsafe environment and act as a conceptual first-stage facilitator to landmine identification. The issue is not just a matter of movement. The ideal rover to do such a task should have a combination of several capabilities at once: it should move in a controlled mode, perceive the obstacles around it, maintain a consistent sense of space and support a detection system within a greater robotic workflow.

A simulated real landmine survey setting is not assured to be laid out or free of obstacles. In even a simplified simulation the rover has to contend with the fundamental problem of autonomous navigation in the presence of environmental features. It implies that the robot must have a perception layer, a frame system, a motion controller and a means of combining sensor data with navigation decisions. If these layers would not be functioning as a team, the rover would be just a manual driven model.

The other aspect of the problem is in localization and motion consistency. Autonomous navigation is particularly reliant on the capability of the robot to estimate its movements. Without the correct reflection of wheel motion in odometry, then mapping, localization, and navigation will start to degrade. This became particularly apparent during the early design phase when the four-wheel variant of the rover generated odometric instability through slippage and discrepancy between the desired and observed motion. That experience made it evident that the problem was not merely one of sensor-related nature but also one of strong dependence on the morphology and the kinematic appropriateness of the robot.

Another systems-integration issue is also taken care of in the project. The platform in a ROS-based robot is not only dependent upon the physical model. The robot should publish and subscribe to the appropriate topics, have a valid TF tree, interface cleanly with simulation plugins, and support controller behavior that matches the actual design of the robot. When at least one of these layers is disjointed, the entire system starts to fail in unobtrusive ways. That is why, the design of robots, their simulation, and integration of the software will be discussed as the same engineering problem and not as different tasks.

1.4 Proposed Solution

This project suggests a **LiDAR-guided differential-drive rover** developed within the **ROS 2 Humble** ecosystem and simulated in **Gazebo Classic** to meet these challenges. The rover is developed as an autonomous mobile robot with core capabilities of environmental sensing, safe movement, and supporting a mine-detection-oriented mission scenario.

The **two wheels of the differential-drive configuration have caster support** and this design was chosen after considering the shortcomings of a previous four wheel effort. The latter design is consistent with popular ROS control assumptions and simplifies the process of simulating,

controlling, and localizing the robot. The first perception input is a LiDAR sensor that helps the rover to scan the surrounding obstacles and meaningfully interact with mapping and navigation systems.

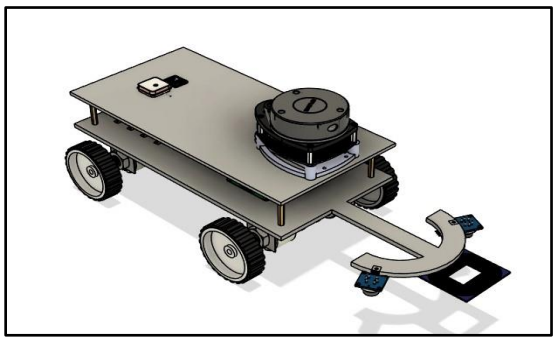

Figure 1: Original Concept

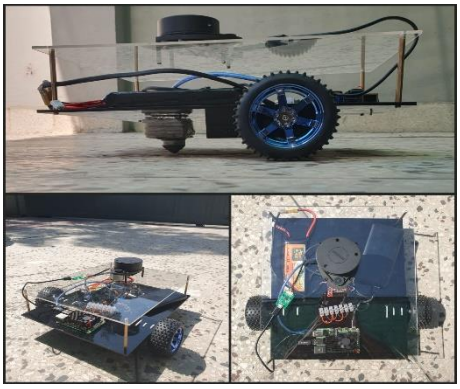

Figure 2: Final Implemented Concept

The software architecture is based on standard ROS 2 concepts. The robot is characterized in **URDF/Xacro**, and has been launched using structured launch files, visualized in **RViz**, simulated in **Gazebo Classic** and integrated with components related to navigation, such as **SLAM** and **Nav2**. The subsystems communicate with each other via ROS nodes, topics, transforms, and parameter files. Such a modular design simplifies the project to debug, extend and document.

Along with mobility and perception, the rover also enables a landmine detection concept in the simulation environment. The project models mines as non-obvious hazards that are not visible but manifest when the rover enters a specific sensing range or satisfies a specified detection criterion. This will enable the project to stay within the reasoning of mine survey and not just mere interaction with visual obstacles. This can then be translated into the ROS system using messages, topics or markers to be visualized and analyzed later.

The suggested solution is not thus a deployable mine clearance robot, but a structured and technically meaningful prototype. It shows how the first robotics simulation could be utilized to construct, test and refine an autonomous rover that was to be used in reconnaissance of hazardous areas.

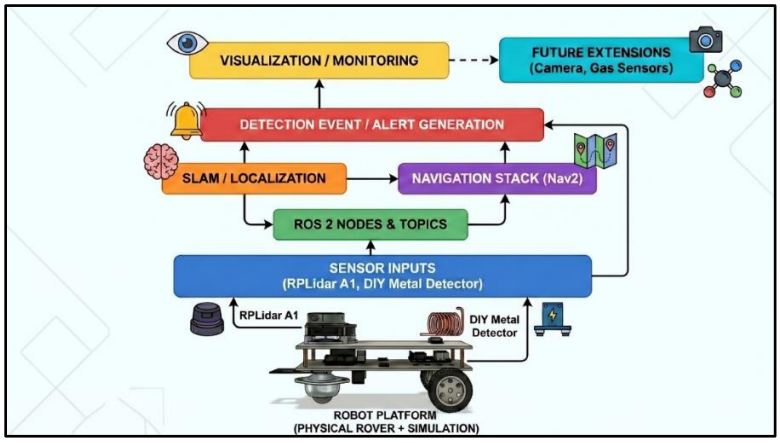

Figure 3: Overall System Concept of the Proposed Rover

1.5 Project Objectives

The primary goal of this project is to design and develop a LiDAR Guided Rover to detect landmines that will work autonomously in a simulated environment using ROS 2 and Gazebo Classic. The primary goal may be broken down into the following more specific goals:

1. To develop a model of mobile rover, which can be autonomously controlled in a ROS 2 environment.

2. To create the description of the robot in **URDF/Xacro** in the proper location and form of joints, links and sensor placement.

3. To connect the rover with **Gazebo Classic** which is used to simulate and test rover physics.

4. To set up LiDAR-based perception to be able to sense obstacles and interact with the environment.

5. To facilitate map creation and environment awareness via a SLAM-ready workflow.

6. To connect the rover with the Nav2 stack to provide autonomous navigation and motion planning.

7. To create a mine-detection-oriented simulation logic in which suspected mine locations can be identified and represented within the ROS ecosystem.

8. To test the impact of rover morphology on the performance of the navigation system, particularly the change in design of the rover to a two-wheel differential drive with caster support.

9. To create an integrated robotics workflow to integrate modelling, simulation, control, navigation, and system integration in a single project.

1.6 Scope of the Project

This project is limited to the design, simulation, and integration of an autonomous rover to be operated in a ROS-based environment in a way that is focused on landmine-detection-oriented operation. The project is primarily aimed at the robotics pipeline, as opposed to actual explosive field sensing or field deployment. The work, as it is now, focuses on simulation, rover architecture, LiDAR-guided motion, and the execution of a conceptual mine detection layer.

The project involves robot modeling, LiDAR integration, ROS topic flow, transform management, controller setup, Gazebo simulation, RViz visualization, and the workflows associated with navigation, which includes mapping and path execution. It also encompasses the analysis of design

trade-offs, most notably the analysis of the transition between 4-wheel layout to the more stable 2-wheel differential-drive layout.

It is not the purpose of the project to provide a field-tested landmine clearance system. It also does not purport to do actual explosive detection, actual world autonomous demining or live neutralization of mines.

1.7 Report Organization

This report has been structured to show the real flow of the project since the concept is developed to the system integration. After this introduction chapter the second chapter is a review of related work in mobile robotics, landmine detectors, LiDAR-guided navigation and autonomous rover development using the ROS framework. Subsequently, the report states the problem in a more organized engineering environment and determines the design requirements of the system that are of key concern.

The following chapters are devoted to the technical implementation of the rover, such as the design of the robot, ROS 2 architecture, URDF/Xacro modelling, Gazebo integration, LiDAR-based perception, SLAM workflow, and configuration of the navigation stack. The chapter also details the design shift to two-wheel systems and four-wheel systems, and how this impacted odometry and system behavior.

The subsequent chapters feature testing, outcomes, discussion, and the lessons learned in the process of development. The report ends with the limitations of the current work and directions of future improvements, in particular, with regards to hardware deployment, coverage planning, multi-sensor fusion, and more realistic logic of minefield inspection.

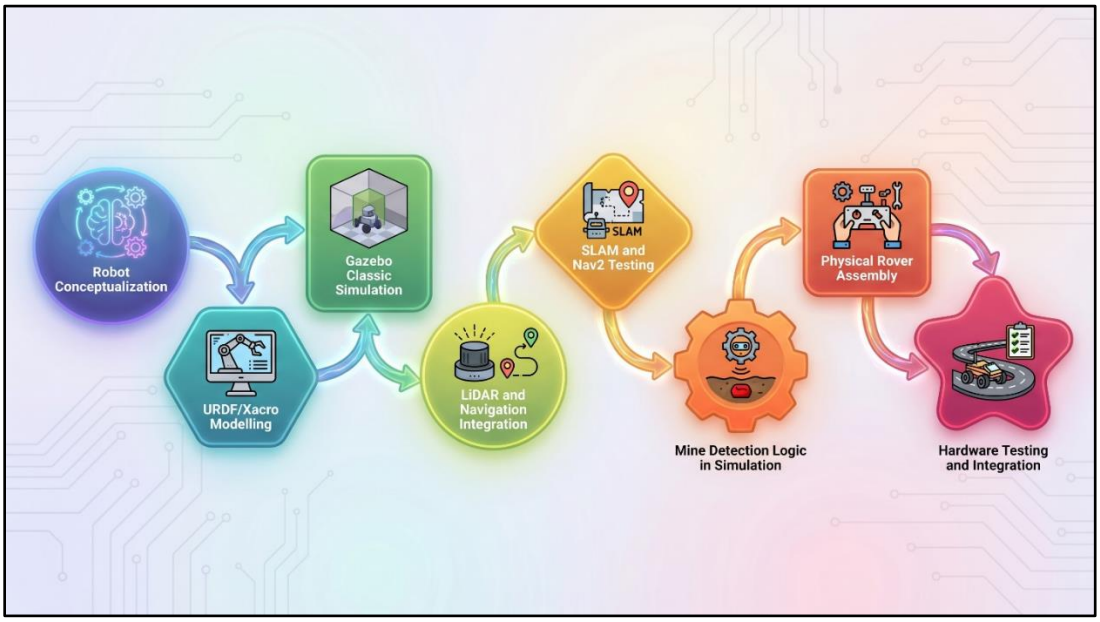

Figure 4: Project Development Workflow

Chapter 2: Literature Review

2.1 Introduction to Related Work

The literature of interest to this project is at the intersection of humanitarian demining, autonomous ground robotics, environmental sensing, and simulation-based robot development. Even though the particular system that was developed in this project is a student scale rover, the concepts behind the development of this rover are based on some of the well-established research directions. These encompass the conventional methods of landmine detection, robotic survey programs, LiDAR-based navigation, SLAM-based mobile robotics and the exploitation of ROS-based software ecosystems to design and evaluate robots.

An examination of the literature available indicates that the problem of landmine detection is not a one-discipline issue. It entails perceiving, movement, engagement with the terrain and interpretation of data, security, and uncertainty-based decision making. Other methods are principally concerned with the sensing mechanism, such as metal detectors, ground penetrating radar or chemical sensing. Others pay more attention to the platform, e.g. unmanned ground vehicles or multi-sensor rovers that will enter the hazardous environment instead of humans. Meanwhile, the mobile robotics field has grown to a mature stage through the development of middleware like ROS, enabling the integration of perception, control, planning, and visualization.

This literature review is not merely aimed at summarizing the work done in the past, but also to place the current project in its right technical context. The review needs to cover both application-oriented research, and system-oriented robotics workflows since the project integrates both simulation-first robotics development, and subsequent physical implementation. This chapter thus analyses the problem in both directions first, how landmine detection has conventionally been done, and secondly, how autonomous rover systems can be designed to encompass safer and more extended survey endeavors.

2.2 Conventional Landmine Detection Approaches

Conventional landmine detection systems have long been based on manual processes being performed by trained deminers with handheld devices [2]. The most popular method is the usage of metal detectors which are still popular due to their low cost, portability and capability to detect metallic objects buried under the soil. Nevertheless, the metal detectors have a significant drawback in that they tend to give a signal to some harmless buried metallic debris like fragments, wires and scrap. This results into false positives and greater time spent undergoing clearance operations.

A second commonly known technique is use of **ground penetrating radar (GPR)**, which tries to detect any disturbance and the presence of any object underground through electromagnetic wave reflection. GPR can give more information about what is below the surface and may be more effective in detecting the mine with low metallic content. Although this is an advantage, GPR systems tend to be more expensive, difficult to interpret, and computationally intensive. This

reduces their applicability to low-cost educational prototypes or lightweight first-stage robotic systems.

Other methods that have been investigated in the literature are thermal imaging, acoustic methods, hyperspectral analysis, explosive vapor sensing and trained-animal-based detection. All of these approaches have possible advantages, yet they also present their own set of limitations: in terms of cost, reliance on the environment, difficulty in calibration, or complexity in deployment. As an example, chemical and gas-based methods can be prospective in the future systems, yet they can be sensitive to the environmental conditions and may be affected by wind, moisture, and soil conditions.

What emerges in the literature is that no single method of sensing is universal to all situations. This is among the reasons why contemporary studies tend to shift towards multi-sensor arrangements. In the frame of the given project, the chosen approach is the purposeful modesty and practicality: a **DIY metal detector** is used to indicate the detection, and the rover itself is based on mobility, environmental awareness, and safe navigation. This is reasonable in the case of a student project as this will ensure that the system will be grounded in a realistic prototype and at the same time it will be relevant to the larger problem of robotic mine survey.

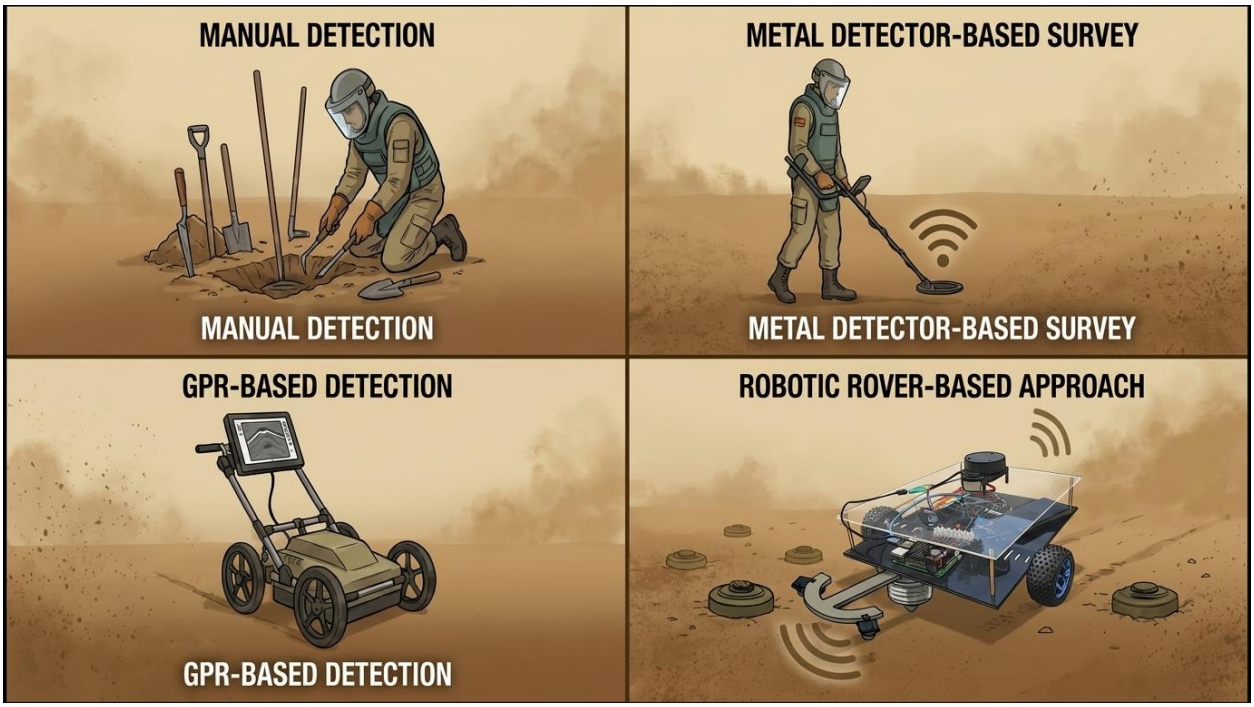

Figure 5: Different Approaches to Landmine Detection

2.3 Robotic and Autonomous Systems for Minefield Applications

The concept of having robots working in dangerous settings is not novel, but it has become much more feasible with the advancement in embedded computing, inexpensive sensors, and robotics

software platforms. Unmanned ground vehicles are enticing in minefield-related applications since they are capable of moving in places, which would otherwise present direct danger to human beings. Their input can be different as per the complexity of the systems. Other rovers can be operated remotely in a process known as remote teleoperation, where the all the movement decisions are made by a human at a safe distance. Others seek greater autonomy, which is letting the robot move around, avoid, and report detection events with reduced human operator intervention.

Teleoperated robots represent a definite safety advantage over simply manually inspecting the terrain, but still rely on the quality of communication, visibility of the terrain, and operator fatigue. Conversely, autonomous or semi-autonomous systems are trying to transfer a part of the decision-making load to the robot itself. These systems are usually based on sensor-based obstacle avoidance, mapping, localization and path planning. Such features are particularly advantageous in cases where the environment is cluttered, partially unknown or difficult to monitor at a remote control station.

In most research applications, the robotic platform is not just a bearer of sensors, but is also the key integration point of perception, motion and mission logic. This is very much related to the philosophy of the present project. The rover that has been developed here is not just a chassis to which a detector is attached. It is a mobile robotic system that must be pose aware and publish transforms and respond to environment geometry, and operate within a hierarchical control structure. It is due to the fact that the project is naturally located within the larger scope of literature on autonomous mobile robots, not necessarily mine sensors.

One of the most important things that were learned during the investigation of robotic minefields is that mobility and detection cannot be considered two separate issues. A detector can perform admirably in isolation, but when the rover loses contact, or drifts off course, the value of the detection data becomes far less significant. This experience proved to be quite pertinent in this project when it came to changing an initial four-wheel concept to the final two-wheel differential-drive design. The literature backs the point that whenever there is a need to work on navigation, localization, or mapping, then the mechanically stable and kinemically predictable motion is a must.

2.4 LiDAR in Mobile Robot Navigation

LiDAR is now one of the most popular sensing technologies in mobile robotics, especially in the context of indoor and semi-structured navigation problems. Its primary benefit is that it can give range information of immediate surroundings with a fair amount of reliability, allowing the robot to estimate positions of immediate obstacles with fairly high reliability. LiDAR is less sensitive to lighting conditions, and the data generated by it are immediately useful in mapping, obstacle avoidance, and costmap generation, unlike camera-only systems.

In the case of the differential-drive rovers using ROS-based navigation pipelines, the LiDAR data is frequently the backbone of the environment’s perception. The laser scans are published on

regular ROS topics and can be consumed by SLAM algorithms, localization systems, collision-checking modules, and the navigation stack. In practice, this implies that only a single LiDAR sensor is required to support multiple layers of rover intelligence simultaneously: it helps the robot feel free space, detect blocked paths, update local costmaps, and refine its perception of the environment.

The **RPLidar A1** was selected as the main ranging sensor in this project. The sensor is used in educational and prototyping applications because the sensor is relatively affordable, easy to interface, and well supported by the ROS ecosystem. The reasons why it is best suited in this project lies not only in the cost, but also in the fact that it is naturally integrated into the simulation and physical implementation workflow. A LiDAR-like sensor can be modeled in simulation with Gazebo plugin, and ROS scan topic. In hardware, the actual RPLidar A1 offers a practical interface between simulation findings and the actual testing of rovers.

LiDAR also has a significant systems role other than obstacle avoidance. Laser scan data is used in ROS-based navigation, either for the **local costmaps** or the **global environment reasoning**. This is the reason, that costmaps are important. They are not merely pretty pictures; they demonstrate that the rover is actually sensing the surrounding world in a significant manner.

2.5 ROS2, Gazebo, and Simulation-Based Robot Development

The current development of robotics is becoming increasingly reliant on software frameworks, which support modularity, reuse, and integration across sensing, control, and planning layers. One of these systems is the **Robot Operating System (ROS)**, which has been significantly influential. As ROS 1 was replaced by **ROS 2**, the developers received better support of modular communication, better flexibility of middleware provided by DDS and more robust architecture of long-term robotics applications. ROS 2 has a structured interface to create entire robot systems with nodes, topics, transforms, launch files, and reusable packages [3].

The simulation has also been at the heart of the contemporary robotics workflows. Most teams now start with digital models of robots, simulated sensors, and virtual test platforms, instead of directly testing all ideas on hardware. The technique minimizes the risk of development, enables the isolation of bugs, and scientists can study the behavior of robots under controlled conditions before they can be physically deployed. In this project, **Gazebo Classic** was taken as the major simulation platform, which works with ROS 2 Humble to validate the modeling of the robots, LiDAR integration, controller behavior and navigation logic before transitioning to the actual rover [5].

Simulation-first method is particularly useful when the application of interest is safety sensitive. Even a simplified simulation environment, in a project associated with landmine detection, offers an engineering advantage of some importance: it will enable the developer to test navigation, obstacle handling, and detection logic without necessarily relying on unstable or incomplete hardware. It also provides a repeatable environment where behavior of systems could be refined in small steps.

The **RViz** role is also significant in the case of such workflows. Whereas Gazebo implements physical interaction, RViz lets you have a glimpse of how the robot perceives the world based on ROS data streams. This encompasses robot pose, LiDAR scans, maps, transforms, planned paths, costmaps and detection-related markers. This is why, the concept of simulation in this project cannot be perceived, as Gazebo activity only. The actual workflow contains Gazebo to perform dynamics, ROS 2 to communicate and control, and RViz to visualize and debug the system.

2.6 Differential Drive Rovers and Odometry Challenges

Design of mobile robots is directly connected with assumptions on control. The mechanical structure of a rover not only determines the ability of the rover to move physically, but also the extent to which the movement of a rover can be described mathematically within a navigation pipeline. Differential-drive robots are particularly popular in pedagogical robotics and research prototypes due to their comparatively simple, well-known and well-supported motion model. Having two actively driven wheels and passive support by a caster or skid element, such robots are frequently easier to model and localize compared with more mechanically complex options.

Nevertheless, even rather basic wheeled robots can exhibit severe odometry errors provided their design does not coincide with the assumptions of the control model. The estimated motion of the rover can be caused to drift off its actual motion due to wheel slippage, uneven wheel contact, poor friction behavior or over constrained motion. This makes mapping, localization and path planning problematic as the higher-level systems rely on odometry being at least reasonably consistent.

This is not just a theoretical issue, as far as the present project is concerned. The team started to investigate a **four-wheel format**, however, during the experiment this design generated clear issues that were associated with **wheel slip** and **odom shift**. The motion of this robot was not as predictable as needed to have stable navigation behavior, especially within the differential-drive assumptions that are common to ROS controllers and workflows based on Nav2. Due to this, the project was reverted to a **two-wheel plus caster configuration**, which offered cleaner motion behavior and more reliable integration with the navigation stack.

This design choice is highly justified by the wider literature on robotics, which repeatedly demonstrates that mechanical simplicity can in many cases be beneficial in improving software stability. A rover that is easier to describe kinematically tends to be easier to localize, easier to simulate and easier to control. In this project the end result of the rover configuration is the direct reflection of the lesson.

2.7 Research Gap and Project Positioning

A review of the literature reveals that much of the existing body of work in mine detection is either overly specialized in its sensing or is large and complex in its robotic nature, which is far beyond the size of a project by a student. Simultaneously, a large number of educational projects involving mobile robots are currently centered on navigation without placing the robot in an application

context of a strong humanitarian interest. This brings in handy space to use in a project such as the one present.

The contribution of this project is not that it introduces a brand-new detection principle or an industrial-grade demining platform. Its value lies in combining several important ideas into one coherent system: a simulation-first development process, ROS 2 Humble integration, Gazebo Classic validation, LiDAR-guided autonomous navigation, a rover architecture inspired by Articubot One, and a later physical implementation using real hardware such as the Raspberry Pi 4, RPLidar A1, L298N motor driver, encoder motors, and a DIY metal detector.

This project therefore occupies a practical middle ground. It is more structured and application-driven than a generic navigation demo, but more achievable and reproducible than high-cost research-grade mine-clearance systems. It demonstrates how modern robotics tools can be used to prototype a rover for hazardous-environment inspection while still remaining grounded in undergraduate-level engineering constraints such as affordability, modularity, and implementation feasibility.

A different engineering gap that is not usually given due attention in simplified robot development can also be noted by the project: the relationship between the morphology of the robot and its reliability when navigating in a specific environment. The project provides a true-to-life lesson to why design decisions impact the localization, odometry, and the overall system behavior. In that sense, the project is not only a mine-detection-themed rover, but also a case study in iterative robotics design.

Table 1: Comparison of technologies used in landmine detection systems

Method / Technology	Main Principle	Advantages	Limitations	Relevance to This Project
Manual metal detection	A trained human operator uses a handheld metal detector to scan the ground for buried metallic objects.	Low equipment cost, simple operation, widely used in real demining practice.	Extremely slow, dangerous for the operator, high fatigue, and frequent false positives due to metallic debris.	Forms the baseline problem that this project aims to improve by reducing direct human exposure through rover-based operation.
Ground Penetrating Radar (GPR)	Electromagnetic waves are transmitted into the ground and reflected back from buried anomalies.	Can detect subsurface disturbances and is useful for identifying low-metal or non-metallic objects.	Expensive, computationally demanding, sensitive to soil conditions, and difficult to interpret in low-cost systems.	Relevant as an advanced alternative, but not selected due to cost and complexity constraints at undergraduate project level.

Camera-based visual inspection	Cameras capture surface imagery for remote monitoring, scene understanding, and potential visual confirmation.	Provides real-time visual feedback, useful for teleoperation, documentation, and environmental observation.	Cannot directly detect buried mines, highly dependent on lighting, and may miss hidden hazards.	Useful as a future extension for real-time operator visuals and remote monitoring support.
Gas / chemical sensing	Sensors detect vapors or chemical traces associated with explosives or buried hazardous materials.	Can complement metal detection and may help identify mines with limited metallic content.	Environmentally sensitive, affected by wind and soil conditions, and usually not reliable enough as a standalone method.	Identified as a future scalability option for expanding the rover into a multi-sensor hazardous-environment platform.
LiDAR-guided robotic rover	A mobile rover uses LiDAR for obstacle perception, navigation, mapping, and autonomous motion in hazardous areas.	Reduces human exposure, supports autonomous or semi-autonomous movement, integrates well with ROS, and allows simulation-first development.	Navigation alone does not confirm buried mines, and the rover still requires a separate mine-indication mechanism.	Directly aligned with the core objective of this project. The rover uses LiDAR for navigation and environment awareness.
Metal detector integrated with rover	A rover-mounted metal detector senses metallic objects beneath the ground while the robot handles mobility and positioning.	Combines detection with robotic movement, reduces need for humans to enter hazardous areas directly, and supports systematic survey.	Still affected by false positives from metallic clutter, and detection accuracy depends on sensor quality and placement.	Directly relevant to the physical implementation of the project through the DIY metal detector mounted on the rover.
Simulation-based robotic development using ROS 2 and Gazebo	Robot behavior, sensors, navigation, and mission logic are first developed and tested in a simulated environment.	Safe, repeatable, cost-effective, and ideal for debugging before hardware deployment.	Simulated behavior may not perfectly match physical terrain and hardware realities.	Central to the project methodology, since the rover was developed in simulation before physical implementation.

Chapter 3: Problem Statement and Objectives

3.1 Problem Statement

The affected landmine areas remain a significant humanitarian recovery challenge, civil safety and engineering intervention. The difficulty does not merely lie in the fact that detection and preliminary inspection must, in most cases, be performed in an environment in which uncertainty is high, and the cost of an error is severe. Survey methods using human operators expose them to direct hazard, whereas sophisticated automated approaches are frequently costly, intricate, and out of the reach of low-cost academic deployment. It presents an engineering challenge to devise a low-cost autonomous rover that will be able to support landmine-related survey work and minimize direct human exposure and demonstrate reliable robotic behavior.

The problem in the context of this project is addressed as a **robotics problem** and a **hazard-response problem**. In the robotics view, the system needs to be able to navigate in a world that has obstacles knowledgeable of it, stable control, and behavior reproducibility. Emerging on the hazard-response basis, the rover has to be capable of supporting a workflow that is oriented towards the detection of landmines, as such that suspected locations of landmines can be identified and reported without the need to send a human being directly into the survey area. These two facets of the issue are closely interrelated. A detector alone is not sufficient when the rover is unable to move predictably, and autonomous navigation alone is not sufficient when the robot has no meaningful mission-specific sensing purpose.

The main issue then becomes how to design and integrate a rover that, once it is put into practice, can support **landmine detection by guiding a rover with LiDAR**. The system should integrate robot modelling and motion control, sensing, communication, navigation and detection logic into a single ROS 2 architecture. As the rover is supposed to provide a transition between simulation and real implementation, the design should also be practical and reproducible with the available hardware resources other than relying on highly specialized research equipment.

The other big component of the problem came up in the very process of development. The first rover design, which was based on a **four-wheel design**, was presented in a mechanically appealing manner, however, it also created problems with the **wheel slip**, **odometry drift**, and unpredictable localization characteristics. Since the project relies heavily on navigation, mapping, and ROS-based mobility control, these inconsistencies directly pose a threat to system reliability. This implied that the engineering problem was not to build a rover, but to build a rover that engineers will agree with, in terms of physical design, the motion model, and the navigation stack. This insight informed the ultimate design decision of a **two-wheel differential-drive rover with caster support.**

In short, the problem that will be the focus of the current project is a requirement of a low-cost, ROS-based, autonomous rover that can navigate with the help of LiDAR and be operated on the principles of mine-detection-oriented operation, transitioning between simulation and physical hardware, and sufficiently stable motion behavior to support mapping, localization, and field-oriented testing.

3.2 Why the Problem Matters

The meaning of this issue is that landmine survey is not only the technically challenging issue, but the socially important one as well. In the actual mine affected areas, even the preliminary inspection and marking activities may cause people to be exposed. Provided that it is possible to employ autonomous rovers to help during these initial stages, then human operators can be kept at a safer distance while still enjoying the benefits of structured environmental data and suspected threat indication.

On an engineering front, the issue also has a bearing in the sense that it has united several fields which are traditionally studied independently. Autonomous navigation, LiDAR perception, description of the robot, ROS middleware and hardware interface are typically taught as separate concepts. Once these ideas are, however, fitted in to a realistic mission-oriented rover, the actual problem then becomes one of integration. A report which talks about a single module in isolation would not portray the actual complexity of the system. The project is important since it shows how the modules relate to each other within a real workflow.

The issue is also applicable regarding educational worth and scalability. A low-cost rover which is initially tested in simulation and then physically implemented can form the basis platform to be extended in many other ways in the future. The future extensions in this project are camera-based real-time visual monitoring as well as gas-sensor integration to give greater hazardous-environment sensing. This implies that the work is not a dead-end prototype; it is a platform which can be extended to a richer multi-sensor robotic system.

3.3 Proposed Project Direction

This project will use a stepwise engineering methodology to solve the identified problem. The initial step is dedicated to the **simulation-based design and validation in ROS 2 Humble** and **Gazebo Classic**, where the rover model, kinematics, sensor integration and navigation behavior can be studied in a controlled environment. The second phase shifts to physical implementation, where the validated ideas are translated onto a real rover platform constructed around a **Raspberry Pi 4, Arduino Nano, Arduino UNO, L298N motor driver, 76 RPM encoder motors, RPLidar A1**, and a **DIY metal detector**.

The rover is intended to be a **differential-drive mobile robot**, as such a design is a pragmatic choice between simplicity, controllability, and compatibility with existing ROS-based navigation workflows. The LiDAR sensor is a sensor of the environment used to map and navigate obstacles, and the detection side of the project is represented by a metal detector mounted on a rover to detect possible buried metallic objects. All these factors enable the rover to serve as a first-stage survey platform instead of being just a navigation demonstration.

The separation of **high-level control** and **low-level motor interfacing** is one of the main architectural approaches of the proposed solution. The Raspberry Pi is used to execute the high-level robot behavior, such as the ROS-based communication and navigation logic. Motor

interfacing is in the meantime done via the **ROS Arduino Bridge** using an **Arduino Nano**, which communicates with the **L298N motor driver** to actuate the encoder motors. This design enables the rover to enjoy the benefits of the ROS-based software flexibility and at the same time enjoy the benefits of simple and practical embedded motor control hardware.

The decision made is thus not only technically sound, but also realistic in the context of an undergraduate final year project. It does not promise a fully field-ready demining robot, but still provides a serious and meaningful prototype with simulation validation, ROS integration, and physical implementation.

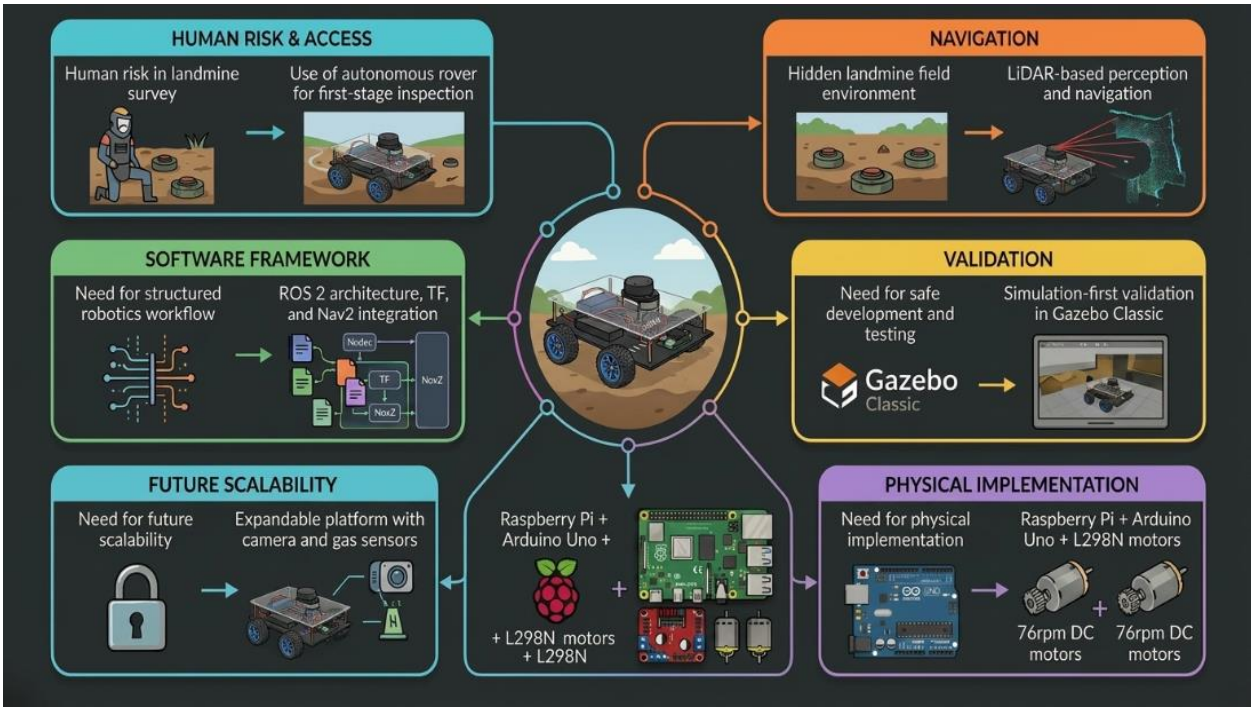

Figure 6: Problem-to-Solution mapping for the proposed rover

3.4 Project Objectives

The primary goal of the project is to design, simulate, implement and evaluate a **LiDAR Guided Rover to detect landmines** that reduces direct human interaction in the initial hazardous-area survey and demonstrates a complete robotics workflow including simulations, hardware and evaluation.

The specific objectives of the project are as follows:

1. To create a rover model that can be used to develop autonomous navigation in a ROS 2 environment.
2. To incorporate LiDAR-based perception to detect obstacles, environmental awareness and navigation support.

3. To apply the simulation-based development in Gazebo Classic and then transition to real hardware.
4. To establish ROS-based communication between sensing, control, and navigation units using nodes, topics, and transforms.
5. To implement a differential-drive rover architecture that is conducive to the stable estimation of the rover motion and navigation behavior.
6. To incorporate a mine-detection-oriented sensing mechanism by using a DIY metal detector in the physical rover.
7. To realize hardware control via a Raspberry Pi 4 with ROS and Arduino Nano through the ROS Arduino Bridge and an L298N motor drive.
8. To assess the effect of rover morphology by comparing the original four-wheel design to the final two-wheel plus caster design.
9. To develop a system base which may be further on extended with camera-based monitoring and gas-sensor-based hazardous-environment sensing.

Table 2: Summary of Project Objectives and associated Design Targets

Objective Area	Objective Description	Design Target / Intended Outcome
Robot platform design	Develop a rover platform suitable for autonomous movement and sensor integration.	A stable differential-drive rover model in simulation and a corresponding physical rover implementation.
Navigation and perception	Use LiDAR data to support obstacle awareness and autonomous movement.	Reliable LiDAR scan acquisition for mapping, obstacle handling, and navigation support.
Simulation workflow	Validate rover behaviour in a virtual environment before hardware deployment.	Functional simulation in ROS 2 Humble and Gazebo Classic with RViz-based visualization.
ROS integration	Build the project using structured ROS communication and robot description methods.	Proper use of nodes, topics, launch files, TF tree, and URDF/Xacro modelling.
Detection capability	Integrate a mechanism for mine-detection-oriented operation.	Simulation-based detection logic and physical integration of a DIY metal detector.
Motion control	Ensure the rover can execute controlled movement in simulation and hardware.	Differential-drive control with ROS-based command flow and physical actuation through ROS Arduino Bridge.
Hardware implementation	Translate the simulation concept into a real rover platform.	Raspberry Pi 4, Arduino Nano, L298N, encoder motors, RPLidar A1, and power system integrated into a working prototype.
Design refinement	Improve system stability through informed mechanical design choices.	Transition from four-wheel concept to two-wheel plus caster design for improved odometry and navigation consistency.
Future scalability	Keep the platform extendable for later enhancements.	Ability to add camera-based monitoring and gas sensors in future work.

3.5 Functional Requirements

There are several functional requirements that the rover needs to meet in order to be regarded as a meaningful implementation of the proposed project.

To begin with, the system should be capable of modeling the rover in the right way within the ROS ecosystem. This implies that the description of the robot, links, joints, transforms and sensor placements should be described in a similar manner. Second, the rover should be capable of moving as a differential-drive robot and responding to motor commands in both simulation and physical implementation. Third, the LiDAR sensor should give usable scan data to interact with the environment, perceive obstacles, and visualize them.

The system also needs to provide navigation-related functionality. This should allow the rover to exist in a simulated world, to be visualized in RViz, and to take part in localization or mapping processes that are applicable to ROS 2 navigation. Also, the hardware rover will have to support communication between the Raspberry Pi and Arduino Nano to execute motion with the help of the Arduino Nano.

Lastly, the rover should be able to accommodate mine-detection-oriented behavior. This, in the case of simulation, implies that the project must reflect the notion of the concealed dangers and the system-wide reaction. In hardware, it implies that DIY metal detector needs to be built as a practical sensing unit that can be used to indicate the potential presence of metallic objects beneath the surface.

3.6 Non-Functional Requirements

Besides direct functional behavior, the project also needs to fulfill a number of non-functional requirements. **Modularity** is one of the most crucial. The rover is not to be developed as a monolithic script or tightly bound design. It ought to be organized in the sense that the robot model, sensor setup, control layer, and navigation layer can be comprehended and edited individually.

Reproducibility is also another important requirement. This being a final year project, the system is supposed to be implementable with available hardware and open-source software tools as opposed to using expensive industrial hardware. This is the reason why the selected architecture makes use of Raspberry Pi 4, an Arduino Nano, Arduino UNO, RPLidar A1, L298N and a DIY metal detector instead of specialized high-cost sensing systems.

Scalability is also needed in the project. Although the existing implementation is that of LiDAR guidance and metal-detection-oriented operation, the system must be able to be expanded. Additional features like a camera to monitor the real time visual conditions and gas sensors to monitor the wider hazardous-environment conditions should be possible without redesigning the entire platform to start with.

Another non-functional requirement is **stability of motion and odometry**. As localization and navigation require the consistency of wheel motion, the robot platform needs to be both mechanically and kinematically appropriate to the selected control mechanism. This need had a direct impact on the shift out of the previous four-wheel concept to the final two-wheel differential-drive concept.

3.7 Constraints and Design Considerations

As with any student project, this work was created under practical conditions. These were constrained budget, constrained development time and the need to use hardware which was available and sufficiently documented. These limitations had a significantly strong impact both on sensor selection and system design. Instead of trying to develop a highly complex mine-detection system, with costly subsurface imaging hardware, the project targeted a demonstrable and manageable combination of LiDAR-guided mobility and a DIY metal detector.

The other limitation was that there was the necessity of balancing realism and feasibility. The full realistic landmine detection platform would entail strong interaction with the terrain, high-quality sensing, field safety measures, and very reliable localization in the outdoor environment. This type of system is well beyond the capabilities of a final year undergraduate project. The design decisions made here therefore focus on educational merit, engineering rightness and practical implementation as opposed to full deployment in the real world.

The development process needed to take into consideration the constraints of the rover kinematics and the interfacing hardware. The four-wheel early design showed that introducing mechanical complexity can introduce new issues of control and localization instead of fixing them. This left one of the most significant design factors as the correspondence between the real rover design and the ROS assumptions in terms of navigation. This resulted in the final design, which was more stable, simpler and easier to incorporate into the entire software stack.

Chapter 4: Development Methodology and System Workflow

4.1 Development Methodology

This project was developed in a staged and iterative approach to project development as opposed to a straight-line construction process. This was required due to the project being a combination of mobile robot modelling, Software integration of ROS 2, sensor-based navigation, and real hardware implementation. A more mechanical and electrical integration hardware-first approach would have made debugging hard and would have increased the risk of spending time on mechanical and electrical integration before validating the core rover behavior. On this basis, the project was undertaken in the form of a **simulation-first approach**, with the progressive transfer of tested ideas onto the actual rover platform taking place.

During the initial stage, attention was given to perceiving the rover as a robotic system within the ROS 2 ecosystems. This involved defining the robot body, wheel arrangement, and sensor placements by **URDF/Xacro**, the configuration of launch files, and the basic structure of the ROS package. After the robot model could be spawned in the simulation, the next thing was to ensure that the robot behaved as expected with respect to mobility. This phase was also crucial since the very design of the rover was still under consideration, and the experience of the preceding four-wheel prototype showed that there were some problems that would be reflected on the final design.

The second step entailed simulation integration within **Gazebo Classic** and **RViz**. At this point, the rover was not only considered as a geometric model but also as an actual robot. All the simulated LiDAR sensor, robot state publishing, TF tree and differential-drive control workflow were tested to ensure that the platform behaved consistently within a realistic ROS environment. This stage enabled the team to monitor rover movement, frame stability, sensor output visualization, and subsequently test the localization and navigation behaviour.

The third stage was concerned with the development of navigation orientation. When the rover was capable of movement and publication of the required data streams, the system was expanded to **SLAM, localization**, and **Nav2-based autonomous navigation**. This allowed the testing of the interpretation of obstacles by the rover, generation of paths, and reaction to the environment. Notably, the practical implications of wheel slip and odometric inconsistency were also identified during this phase, which further justified the rationale of abandoning the previous four-wheel design and adopting two-wheel differential-drive system with caster support.

The last step was actual physical construction of the rover with actual hardware. The simulation work was not thrown away at this point, but rather it was used as the foundation of the hardware build. The physical rover was equipped with a **Raspberry Pi 4 (8 GB)** as the main ROS computer, an **Arduino Nano** as the low-level motor interface via the **ROS Arduino Bridge**, an **L298N motor driver**, **76 RPM encoder motor**, **RPLidar A1**, **3S 11.1 V LiPo battery**, **a power bank for the Raspberry Pi**, and a **DIY metal detector**. This stepwise development process meant that the majority of the system-level assumptions had already been tested in simulation by the time that the rover was actually assembled.

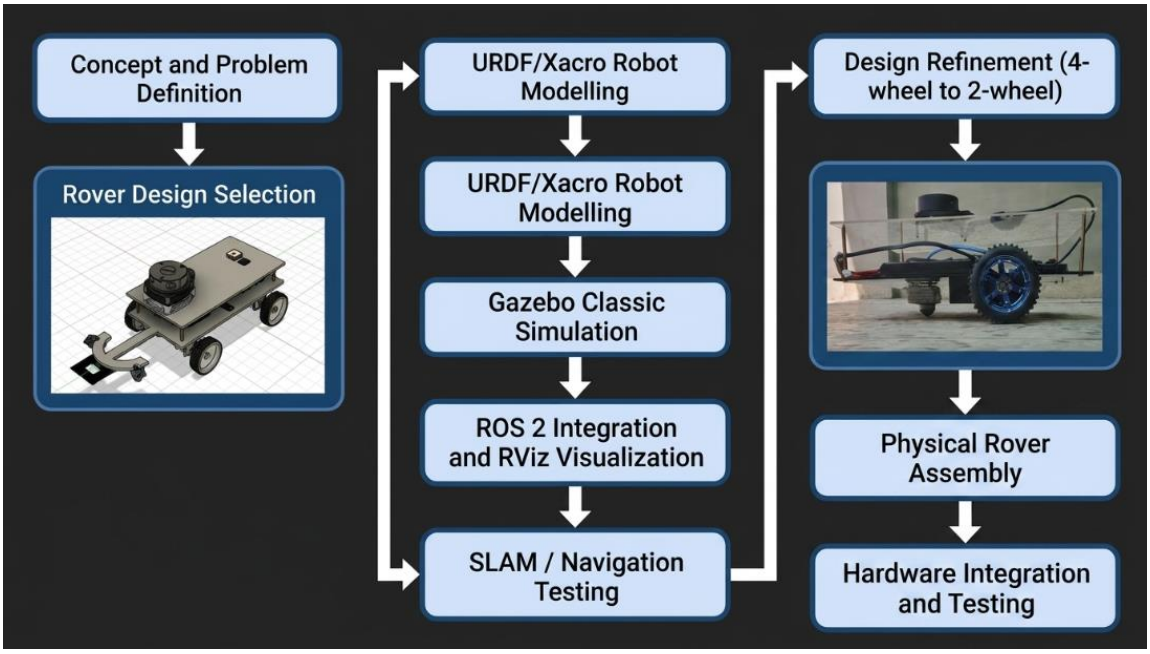

Figure 7: Development Methodology adopted for this project

4.2 Overall System Workflow

The entire project lifecycle can be conceptualized as a pipeline with the start being the design of a robot and the end being a physically produced autonomous rover. The workflow did not divide simulation and hardware into different projects; but instead saw them as consecutive steps in a single development process. The simulation environment allowed a safe and repeatable environment to validate the model, with the hardware phase providing practical evidence that the whole concept could be implemented in a real rover.

At the top, the workflow started by **conceptualizing robots and choosing their kinemics**. This involved the choice of rover morphology, the layout of the wheels, the location of the sensors, and the overall structure of the system. Early design work also featured a four-wheel rover concept, but, after testing and analysis of motion behavior, it was found that this design added unnecessary complexity and odometry-related problems. This resulted in the final design based on adopting a two-wheel differential-drive rover, which is supported by a caster wheel.

Once the mechanical concept had been defined, the rover was modeled in software with **URDF/Xacro**. This step defined the digital representation of the robot needed to integrate with ROS 2. It enabled the team to specify links and joints, chassis geometry, and wheel placement, and sensor frames in a manner that could be consistent across simulation and visualization tools.

The second step consisted in **Gazebo Classic simulation** and **system observation done in RViz**. The movement of the rover, LiDAR behavior and transforms and overall interaction with the environment were analyzed in simulation. This phase also helped in integrating software modules related to navigation such as mapping, localization and obstacle-aware motion. When these functions were beginning to behave reasonably in simulation, the project started to advance to physical implementation.

The last steps of the workflow were dedicated to hardware implementation and lab tests. The conceptual rover logic devised in simulation was transferred to the physical structure of the rover. ROS was run on the Raspberry Pi, motor actuation was provided using the Arduino Nano and L298N, LiDAR data was derived using the RPLidar A1 and the DIY metal detector introduced the mine-detection-oriented sensing capability of the project. This achieved a whole workflow between concept and implementation instead of a fragmented set of sub systems.

Table 3: Major Development Phases followed during the Project

Phase	Main Focus	Outcome
Phase 1	Problem understanding and rover concept selection	Defined the project direction and selected a rover-based mine-detection approach
Phase 2	Robot modelling using URDF/Xacro	Created the structural digital representation of the rover
Phase 3	Gazebo Classic simulation setup	Enabled rover spawning, environment interaction, and initial behavior testing
Phase 4	ROS 2 integration and RViz-based observation	Verified communication, transforms, visualization, and sensor data flow
Phase 5	SLAM and navigation-oriented testing	Evaluated rover movement, environment interpretation, and navigation readiness
Phase 6	Design refinement from 4-wheel to 2-wheel rover	Improved odometry consistency and control suitability
Phase 7	Physical hardware assembly	Built the rover using Raspberry Pi, Arduino Nano, L298N, motors, LiDAR, and detector
Phase 8	Hardware interfacing and testing	Connected sensing, control, and actuation subsystems into a working prototype

4.3 Software and Development Environment

The software platform adopted in this project was selected to facilitate the development of modular robotics, testing of the system by simulation, and subsequent hardware assembly. The main software framework was **ROS 2 Humble** that gave the system a backbone in communication. The rover could be structured through a collection of nodes that communicate with each other by exchanging information in the form of topics, parameters, transforms, and launch files through ROS 2. This modularity was necessary since the system could not afford to become an unstructured codebase.

Gazebo Classic was the main platform of simulation. Gazebo was chosen due to its good support in integrating with ROS-based robot workflows, and its ability to test physics-based movement, sensor plugins, and robot-environment interaction. In this project, Gazebo was not only used to visualize the rover, but also to test wheel motion, rover behavior, and how the geometry of the environment affects navigation.

URDF/Xacro was employed to describe the robot and to model its structure. This enabled the robot to be specified in a reusable and scalable manner. Rather than constructing a rigid, unalterable, description of a robot, Xacro macros allowed a description to be manipulated and changed into any shape required. This particularly came in handy during the process of perfecting the rover design.

The primary visualization and debugging tool was **RViz**. Although Gazebo was used to manage the physical simulation environment, RViz was used to enable the team to view the robot state as viewed through the ROS perspective. This involved watching the rover model, LiDAR scans, maps, transforms, paths and then the behavior of costmaps and navigation output. When it came to validating the fact that the robot was reading the environment properly and not just going in simulation, RViz became particularly critical.

To create code and organize packages, a typical ROS 2 workspace was platformed on the Raspberry Pi and development system. The development process included Python-based launch files, configuration files (written in YAML), and conventions of ROS packages. The hardware component also included the **ROS Arduino Bridge**, with which the Raspberry Pi communicated with the Arduino Nano to send motor commands and to receive low-level sensing-related information.

Table 4: Software Tools and Development Environment used

Tool / Platform	**Purpose in the Project**
ROS 2 Humble	Main robotics middleware used for nodes, topics, launch files, transforms, and overall system integration
Gazebo Classic	Simulation platform used for physics-based rover testing and virtual environment interaction
RViz	Visualization tool used for observing robot model, LiDAR scans, maps, transforms, and navigation data
URDF / Xacro	Robot description framework used for defining links, joints, dimensions, and sensor placement
YAML Configuration Files	Used for controller, navigation, and ROS parameter configuration
Python Launch Files	Used to start and organize ROS nodes and simulation components
ROS Arduino Bridge	Used to connect ROS running on the Raspberry Pi with the Arduino Nano for motor and sensor interfacing
Arduino IDE	Used for programming and uploading code to the Arduino Nano/Arduino UNO
Ubuntu 22.04	Execution environment for ROS on the Raspberry Pi
RPLidar ROS Package / Driver	Used to interface the RPLidar A1 with the ROS ecosystem

4.4 Simulation-First Design Approach

One of the biggest methodological choices that were made in this project was that we would start with simulation and then construct the physical rover. This was a critical decision, both technically and practically. Simulation technically enabled it to be possible to test rover behavior without immediately having to deal with the uncertainty of hardware power problems, wiring mistakes, or mechanical instabilities. In practice, it enabled the project team to iterate faster and safer as well as understand the impact of each ROS component on the overall system.

The simulation-first approach enabled the rover to be validated in layers. To start with, the team could verify whether the robot model was structurally sound. At this point the possibility of checking whether the wheels move as required, whether the LiDAR frame is correctly set relative to the rover, and whether the TF tree is consistent when the motion occurs was a possibility. It was not until those foundations were operational that it made sense to proceed towards mapping, localization and autonomous navigation.

This style of development was also directly connected to finding out the design flaws at the initial stage. The previous four-wheel rover design might have seemed satisfactory in purely mechanical terms, but in a simulation the flaws of the design became apparent in terms of navigation and odometry. The slippage and odom-shift problematic phenomena identified during testing would have been a lot easier to analyze in simulation than it would have been had the project been started directly on hardware. Simulation in that sense did not simply save time; it actually enhanced the quality of designs.

A second key benefit of simulation-first development was that it was possible to generate evidence of debugging. Since the system was operated in ROS and Gazebo, the team could view scans, transforms, rover pose, and navigation behavior in RViz. Such observations were not only useful during development but also in future they were used to make documentation.

4.5 Transition from Simulation to Physical Rover

After the rover design and software architecture had attained a working level in simulation, the project shifted towards actual implementation. This stage was not merely a question of putting the parts together. It involved determining the way the high-level ROS logic would be connected to low-level motor control hardware and how the simulated ideas of sensing would be mapped onto the real devices.

The physical rover did not change the underlying architectural thought that had been developed in the simulation stage. The Raspberry Pi 4 was the main onboard ROS computer, which runs the necessary ROS nodes, communication processes, and system logic. The Arduino Nano was the low-level microcontroller interface used to drive the motor drivers of the L298N using the **ROS Arduino Bridge**. This role separation helped to keep the system organized: the high-level software and integration was placed on the Raspberry Pi, and time sensitive low level motor interfacing was handled via the Arduino.

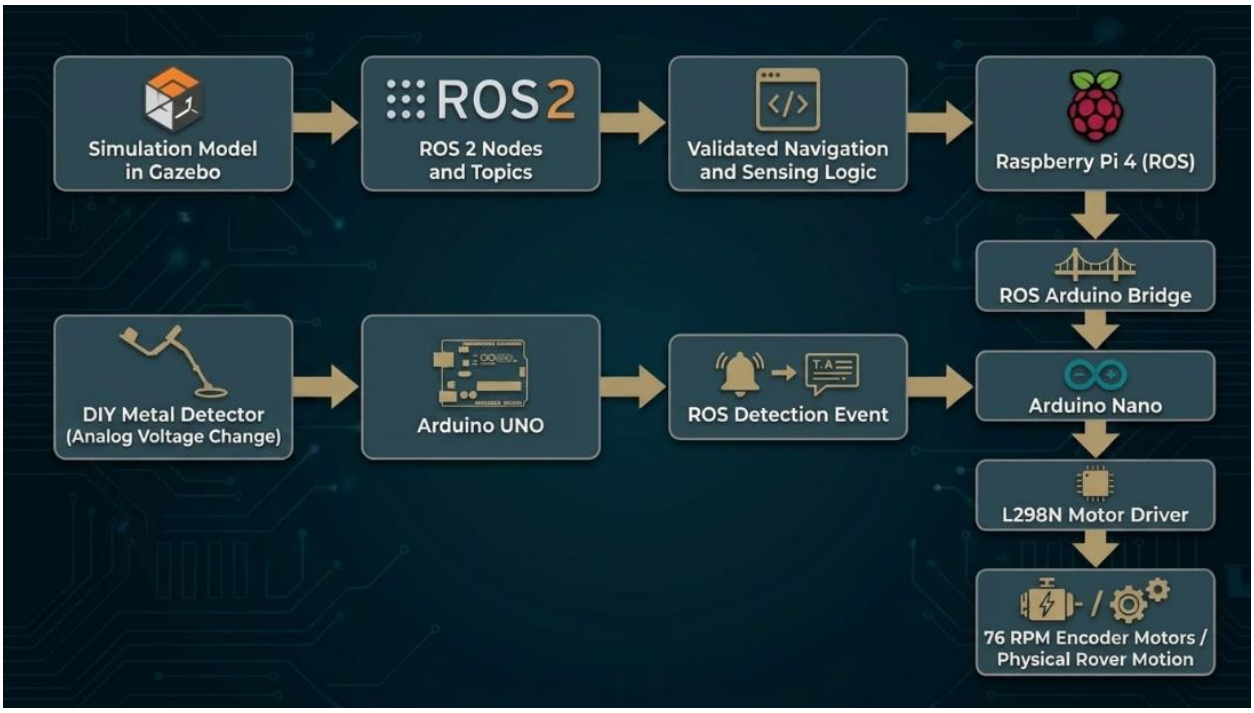

Figure 8: Workflow from Simulation-based Development to Hardware Implementation

The rover employed **76 RPM encoder motors**, which were suitable to controlled rover motion instead of overly aggressive wheel speed. The **RPLidar A1** was attached as the main environmental sensor, and the **DIY metal detector** was used to provide the mine-detection-oriented sensing ability to the project. A **3S 11.1 V LiPo battery** was used to power the rover system, with a power bank being used to power the Raspberry Pi onboard to account for any load fluctuations.

The transition of simulation to hardware was thus not a re-design, but a conversion of tested system ideas into an actual platform. The previous simulation activity had narrowed the gap of uncertainty in the behavior of the robot and enabled the team to base the hardware phase on integration, wiring, mounting and real-world verification as opposed to conceptual debugging in the first instance.

4.6 Sensing and Control Workflow

The sensing and control process of the rover integrates both perception-based navigation and detection-oriented sensing. In terms of navigation, the rover is largely reliant on LiDAR data. The RPLidar A1 scans the surroundings continuously and provides range information that could be used to provide environmental awareness, obstacles management, and subsequent functions related to navigation within the ROS framework. The same principle of perception finds its reflection in simulation in the LiDAR plugin and scan topic organization.

In terms of control, system logic at the high level is executed on the Raspberry Pi with ROS. The ROS environment produces motion commands, which are sent via the ROS Arduino Bridge to the

Arduino Nano. The Arduino subsequently drives the L298N motor driver which turns the left and right encoder motors. This has provided an apparent chain of control between ROS-level decision making and real rover actuation.

The mine-detection-oriented sensing workflow is taken along an alternative yet related route. The principle of the **DIY metal detector** is just the interaction of the magnetic fields around the search coil. When any metallic object enters the effective sensing region, a change in the magnetic flux occurs and therefore leads to a change in the detector output voltage. This output is read by the **Arduino UNO as an analog signal**. The Arduino puts the signal into the detection state by comparing the signal with a threshold or detection criterion and then communicates the detection state to the ROS environment on the Raspberry Pi. This way, the Arduino is not just a motor interface but also the gateway through which the low-level analog detection data becomes a part of the ROS-based decision workflow in the rover.

This architecture is practical in the sense that it is a realistic characterization of the shared responsibilities. The Arduino manages low-level signal reading/writing and control interfacing, whereas the Raspberry Pi manages system-level ROS integration, visualization, and higher-level behavior. This enables the rover to be easily expanded in future, especially when other sensors like a camera or gas sensors are added later.

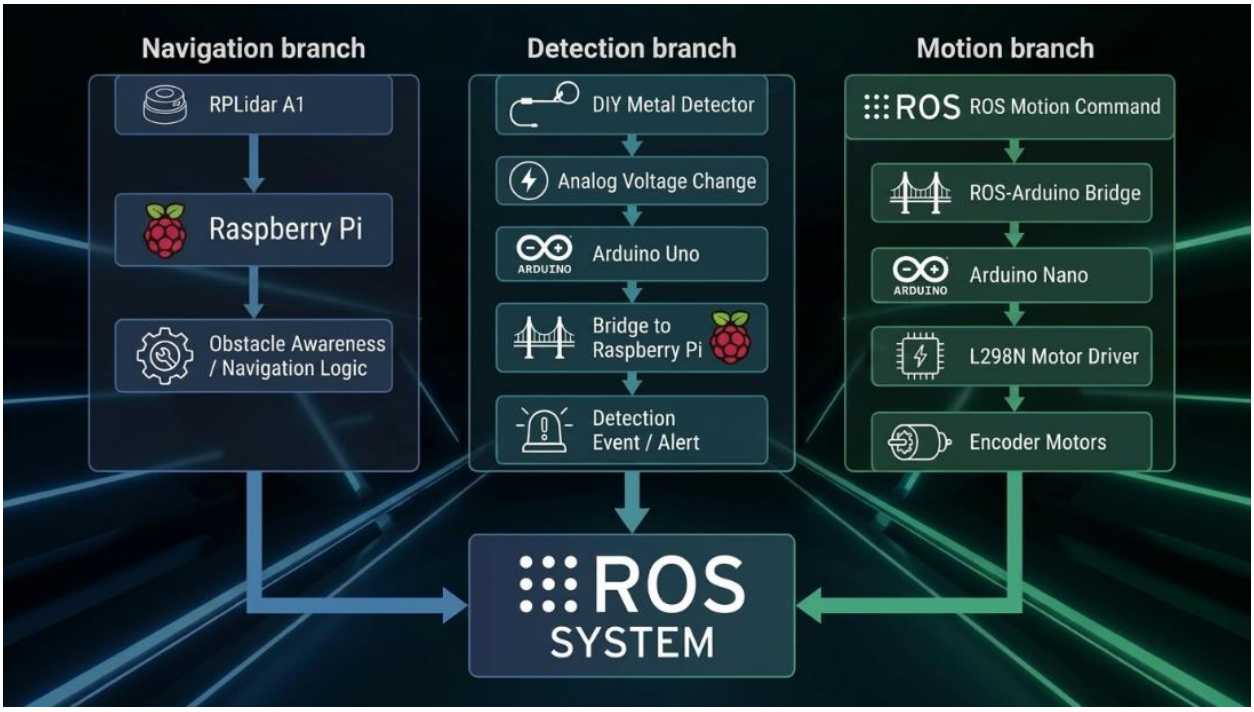

Figure 9: Sensing and Control Architecture of the Physical Rover System

4.7 Summary

This chapter outlined the approach to development and workflow of the LiDAR guided rover to detect landmines. The project was based on an iterative simulation-first methodology, starting with modelling of the rover, integration of ROS, validation through Gazebo, and finally towards the physical implementation of the rover. This methodology enabled the detection of major problems, such as the slippage of wheel and the movement of the odometry, to be detected at an early stage, which directly explained why the decision to change the concept, which was four-wheel drive, to a more stable two-wheel differential-drive concept was made.

The chapter further described the overall system workflow of how simulation and hardware is part of the same continuous development pipeline. The project was based on ROS package-based development which includes ROS 2 Humble, Gazebo Classic, URDF/Xacro, RViz, and ROS package-based development. Physically On the hardware side, the physical rover included a Raspberry Pi 4, Arduino Nano, Arduino UNO, ROS Arduino Bridge, L298N motor driver, encoder motors, RPLidar A1, power system, and a DIY metal detector. Lastly, the sensing and control workflow was also described to demonstrate how LiDAR navigation and analog metal detection were incorporated into a single rover architecture.

Chapter 5: Robot Design, ROS2 Integration, and Hardware Architecture

5.1 Overview of the Rover Design

The rover that was developed in this project was designed as a small autonomous ground vehicle that was to support the landmine-detection-oriented operation by a combination of mobility, LiDAR-based navigation, ROS 2 implementation, and onboard sensing. The two practical needs informed the design philosophy. First, it needed to be a simple enough rover to model, simulate, build, and debug within the confines of an undergraduate final year project. Second, it had to be technically meaningful so as to prove a complete robotics workflow between simulation and physical implementation.

On a higher level, the rover is made up of a differential-drive mobile base, an onboard processing unit, a LiDAR sensor to provide navigation and obstacle awareness, and a DIY metal detector to indicate the potential presence of buried metallic objects. The system has a high-level intelligence, residing on the **Raspberry Pi 4** running the ROS environment and establishing the software-side communication and behavior of the rover. The interfacing low-level motion is handled through an **Arduino Nano** via the **ROS Arduino Bridge**, which enables commands and sensing information related to the motor to move between the embedded control layer and the ROS system.

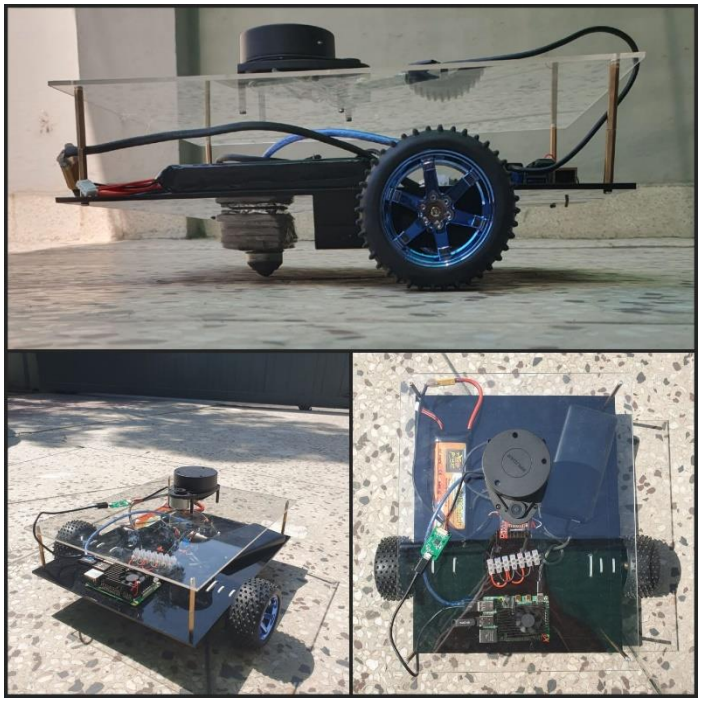

Figure 10: Final physical rover prototype from multiple viewing angles

The physical platform was not intended to be a generic hobby robot but was a rover with a well-defined application context. This not only influenced the positioning of hardware but also the priorities of architecture. The RPLidar A1 was forced to be placed in such a manner that preserved a useful field of view to use in the navigation. The metal detector needed to be near the ground to

be significant in detection behavior. The processing and power elements were required to be configured such that the rover could be wired, it could be made stable to operate, and it could be expanded to add additional features later such as a camera and gas sensors.

The resulting design is thus a tradeoff between implementability and practicality, and architecture and structure on the system level. It is not merely a robot made from available pieces, but a specifically arranged platform, whose mechanical, electronic and software layers were selected to co-operate within the ROS 2 robotics ecosystem.

5.2 Mechanical Configuration of the Rover

The final implemented rover was a **two-wheel differential-drive chassis on caster support**. The choice of this mechanical design was motivated by the fact that it provided more predictable motion behavior than the previous four-wheel concept and the assumptions of the common differential-drive control models used in ROS-based mobile robotics. In this arrangement the two dominant wheels are involved in providing propulsion and steering by controlling the differential velocity and the passive caster provides stability and weight support without restricting the motion.

This option was not only significant in terms of physical simplicity but also in terms of software reliability. Robots based on the differential-drive are some of the most supported mobile platforms by ROS and navigation pipelines due to their relatively easy to model and well understood kinematics. This is very important in the case of a project which relies on odometry, transform consistency, simulation behavior and subsequent navigation. A mechanically simple and kinematically consistent rover is straightforward to simulate, as well as easier to control in both software and hardware.

The rover has **76 RPM encoder motors** that were a better fit to this task than high-speed motors. As the idea of the platform is not a high-throughput but a controlled motion in a detection-oriented environment, moderate-speed motors were more appropriate. A reduced wheel speed is facilitated by improved stability of motion, easier observation in the course of testing, and more viable rover handling during navigation and sensor experiments.

The LiDAR unit was mounted in a higher position on the body of the rover to prevent the chassis to obstruct the scan, and to allow a better horizontal scan of the immediate surroundings. The DIY metal detector was placed in a lower and closer location to the location of interest, as the purpose of the DIY metal detector is related to subsurface metal indication rather than broad-area perception. This difference in mounting location is due to the differing sensing functions of the two devices: the LiDAR watches what is going on around the rover, and the metal detector is focused on the area under or directly adjacent to the rover’s route.

Kinematic and sensing considerations thus dictated the mechanical layout. The setting of the wheel configuration, sensor placement, and physical balance were all chosen in a manner that favored the intended navigation-and-detection workflow and did not consider the individual mechanical decisions to be made.

5.3 Hardware Components and Specifications

The physical rover was constructed based on commercially available hardware components chosen based on factors such as affordability, availability, documentation support and ease of integration into a ROS-based system. This was not aimed at creating a specialized industrial robot, but to create a reliable academic prototype capable of demonstrating real system integration, without exceeding the constraints of final year project.

The primary onboard processing unit was the **Raspberry Pi 4 (8 GB)**. It was in charge of managing the ROS environment, communication, and the higher-level reasoning of the system [9]. As the low-level embedded controller to interface motor related signals and provide the communication path between the hardware layer and the ROS system, the Arduino Nano was used [10]. The electrical interface needed to propel the encoder motors based on the control commands generated using the Arduino path was provided by the L298N motor driver [11].

Table 5: Major Hardware Components used in the Physical Rover Implementation

Component	**Model / Type**	**Role in the System**
Main processing unit	Raspberry Pi 4 (8 GB)	Runs ROS 2, manages high-level software, communication, and system integration
Low-level controller	Arduino Nano/UNO	Interfaces motor control / analog detector reading through ROS Arduino Bridge
Motor driver	L298N	Drives the left and right DC encoder motors
Drive motors	76 RPM encoder motors	Provide controlled differential-drive motion and wheel feedback capability
LiDAR sensor	RPLidar A1	Provides range data for obstacle awareness, navigation, and environment sensing
Metal detector	DIY metal detector	Indicates the presence of buried metallic objects through analog voltage variation
Main rover battery	3S LiPo battery (11.1 V)	Supplies power to the rover's actuation and main electronics chain
Raspberry Pi power source	Power bank	Provides stable power to the Raspberry Pi independent of motor load fluctuations
Chassis	Laser-cut Acrylic Sheet	Supports the mechanical assembly of wheels, sensors, processing, and power components

The encoder motors on the rover were **76 RPM motors**, which were quite suitable to control low-speed mobile operation. In the case of environmental sensing, the platform was an RPLidar A1 which was the primary ranging sensor in obstacle perception, navigation support, and to continuity simulation to hardware. In the case of mine-detection-oriented behavior a DIY metal detector was mounted on the rover. This detector was an analog signal change that could be read by the Arduino UNO and sent into the ROS environment via the bridge architecture.

A **3S 11.1 V LiPo battery** and a **separate power bank** provided power to the **Raspberry Pi**. This isolation was used to make sure that the onboard processing system had a consistent power supply even at times when current fluctuations in the motors would occur elsewhere in the rover. The general layout of the hardware is indicative of a modular embedded robotics design where sensing, processing, actuation and power subsystems each have a different role.

5.4 Electronic and Control Architecture

The electronic and control architecture of the rover is based on a layered design; whereby high-level and low-level duties are explicitly divided. This is a significant design choice as it does not overload a single device with all the tasks and makes the system more easily understandable, testable, and extendable.

The **Raspberry Pi 4** is the base of the computational unit of the rover. It executes ROS, handles the corresponding software packages, and is at the heart of the system-level intelligence of the rover. The Pi is also connected to the **RPLidar A1** to get LiDAR data and the **ROS Arduino Bridge** to communicate with the **Arduino Nano**. The Raspberry Pi is the place where navigation-related logic, visualization, command generation and system integration converge.

At the bottom, the **Arduino Nano** plays the role of a connector between the ROS system and the motor/sensing hardware. The bridge interprets motion-related commands that are sent by the ROS side and uses them to drive the **L298N motor driver** which drives the left and right encoder motors. The **Arduino UNO** has the responsibility of reading the analog signal of the DIY metal detector. When an analog change is detected by the detector through a change in voltage as a result of the presence of nearby metal, the Arduino reads the analog change and can broadcast the resulting detection condition into the ROS environment on the Raspberry Pi.

This architecture is practically sensible due to a number of reasons. Firstly, the Arduino can be easily configured to directly interface with signal-level devices and provide simple threshold-based processing. Secondly, Raspberry Pi is more appropriate to run ROS, package-level logic, high-level robotic coordination. Thirdly, maintainability is enhanced by the separation. The issues of motor actuation, detector reading, and ROS integration can be debugged in a more understandable way when the architecture is reflective of a reasonable distribution of responsibilities.

The general control chain can be then understood in the following way: ROS works on the Raspberry Pi, the decision or motion commands are transferred using the ROS Arduino Bridge, the Arduino Nano is connected to the L298N and motors to provide physical motion and the analog information related to the detector is read on the Arduino side and is made available to the ROS system to provide system-level response.

Table 6: Functional Breakdown of the major rover subsystems

Subsystem	Main Function	Key Associated Components
Processing subsystem	Runs ROS 2 and high-level rover logic	Raspberry Pi 4
Motion control subsystem	Converts control commands into motor actuation	Arduino Nano, ROS Arduino Bridge, L298N
Locomotion subsystem	Produces rover movement through wheel actuation	76 RPM encoder motors, wheels, chassis
Navigation sensing subsystem	Perceives surrounding obstacles and free space	RPLidar A1
Detection subsystem	Senses nearby buried metallic objects	DIY metal detector, Arduino UNO analog input
Power subsystem	Supplies electrical power to the rover electronics and computing hardware	3S LiPo battery, Power bank
Mechanical support subsystem	Physically holds and organizes the rover structure	Chassis, mounts, wheel assembly, caster

5.5 ROS2 Integration and Software Structure

The rover was built around the **ROS 2 Humble** ecosystem, which served as the unifying software framework for the project. This choice was important because the system was not limited to low-level motor movement. It had to support robot modelling, sensor integration, simulation, visualization, navigation, and later physical interfacing. ROS 2 provided the modular communication structure needed for that level of system organization.

Within this architecture, the rover exists as more than a physical machine. It also exists as a robot description, a transform tree, a collection of nodes, and a topic-level communication structure. The robot model was defined through **URDF/Xacro**, allowing the links, joints, sensor frames, and physical relationships of the rover to be described in a structured and reusable way. These descriptions formed the basis for both simulation and visualization workflows.

The ROS software structure also enabled the rover to connect simulation and hardware meaningfully. In simulation, the rover could publish transforms, receive motion commands, and generate LiDAR data in a controlled environment. In hardware, the same ROS-centered philosophy remained, except that the real sensor and actuator interfaces replaced their simulated counterparts. This continuity between simulation and implementation is one of the strongest aspects of the project.

Another significant factor of ROS integration is the clarity of communication. Since the rover has separate layers to enable LiDAR sensing, motion control, detector interfacing, and system-level logic, the ROS structure simplifies the task of organizing those tasks without collapsing all the responsibilities into a single script. It also simplifies the system to be reported in an academic report. Other concepts including **nodes, topics, launch files, TF**, and **ROS-Gazebo integration** are not merely jargon in this project but in actuality, are the main focus in how the rover will operate.

5.6 Design Evolution from 4-Wheeled to Differential Drive

The change in the concept of the initial **four-wheel** rover into the final **two-wheel differential-drive rover with caster support** was one of the most significant engineering decisions in the project. This was not done due to cosmetic reasons. It came about through testing, observing and the practical realization that the previous design would bring about a behavior that was hampering the navigation objectives of the project.

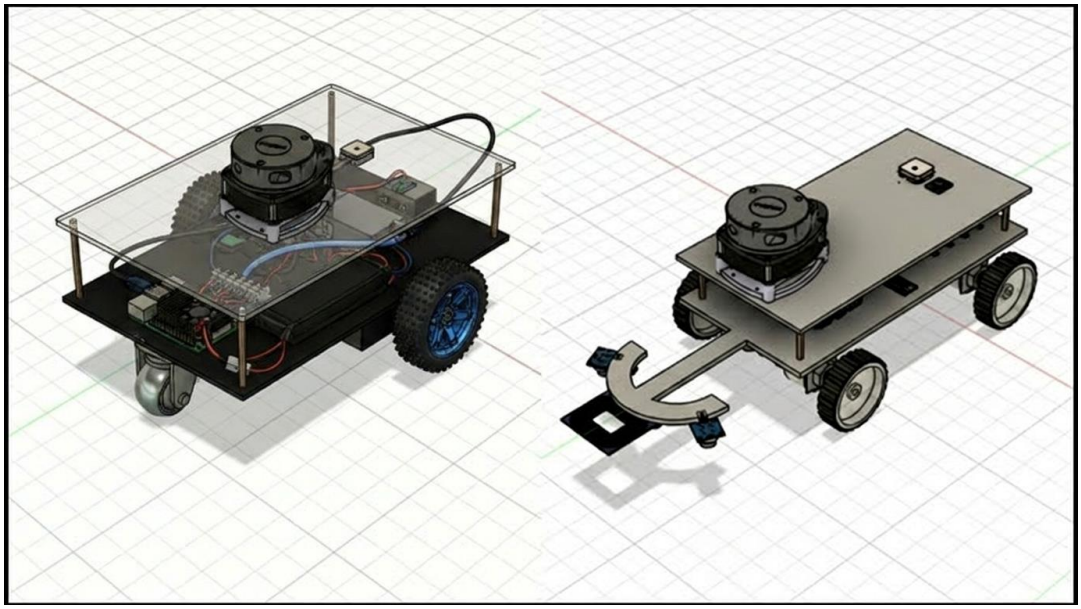

Figure 11: Design evolution from initial four-wheel rover to final differential-drive configuration

A four-wheel rover is a good-looking vehicle at first, since it would seem to be a mechanically stable vehicle, capable of providing superior support over rough terrain. But in this project, the four-wheel design posed some issues with regards to **wheel slip** and **odometry drift**. These issues impacted consistency of the motion and caused the behavior of the rover to be less predictable during the tests related to the motion. As the project relied on ROS-based movement, localization, and operation consistent with a map, these issues proved hard to ignore.

It was not just that the rover was overpowered in wheels. The underlying issue was that the four-wheel layout was not as well-aligned with the assumptions of a differential-drive which the software and control stack were making. This discrepancy caused behavior that caused odometry to be less reliable and motion to be less reproducible. That was a severe shortcoming in a platform that was supposed to be able to support LiDAR-guided navigation and detection-oriented operation.

By re-introducing a **two-wheel plus caster** system, much of this was solved by simplifying the motion model. This physical behavior of the rover became more aligned with usual differentials drive control, which then became more compatible with the ROS navigation workflows. This change also resulted in a rover being easier to reason about, easier to simulate, and easier to implement physically.

This change in design is significant as it shows that the development of robotics is not associated with the desire to add something or select what seems to be stronger in its mechanics. It is concerned with making physical design, assumptions in kinematics, and software control architecture complement each other. In that regard, one of the biggest design refinements of the project was the move to a differential-drive rover.

5.7 Summary

This chapter introduced the rover in the lens of design, architecture, hardware, and integration of ROS. The final platform also demonstrated to be a structured mobile robotic system instead of being a mere assembly of parts. The chapter justified the choice of the differential-drive design of the rover, described the key hardware components used in the physical implementation, and explained the layered control architecture implemented around the Raspberry Pi, Arduino Nano, Arduino UNO, ROS Arduino Bridge, and L298N motor driver.

The role of ROS 2 in the system organization and the design process of the two-wheel rover was also mentioned in the chapter. These aspects, in combination, form the technical base of the following chapter which will deal with the simulated environment, navigation workflow and detection-oriented operation of the rover.

Chapter 6: Simulation Environment, Navigation, and Landmine Detection Workflow

6.1 Simulation Environment Setup

The system was developed and tested in a simulated environment with **Gazebo Classic** and **ROS 2 Humble** before being implemented in hardware as a rover. In this way, the verification of the rover model, sensing behavior, and navigation workflow in a controlled environment was achieved. Simulation was used to minimize the risk of development, since it was possible to repeat tests without necessarily relying on the physical hardware and its stability, the properness of its wiring, and the reliability of the batteries it uses.

The simulated environment comprised the rover model, a navigable world, and the necessary ROS interfaces to navigate the world and to control the rover. Gazebo spawned the robot using ROS launch files, and RViz was used to visualize the robot state, scan data and information relating to navigation. This configuration gave the base needed in the future mapping, localization and autonomous navigation experiments.

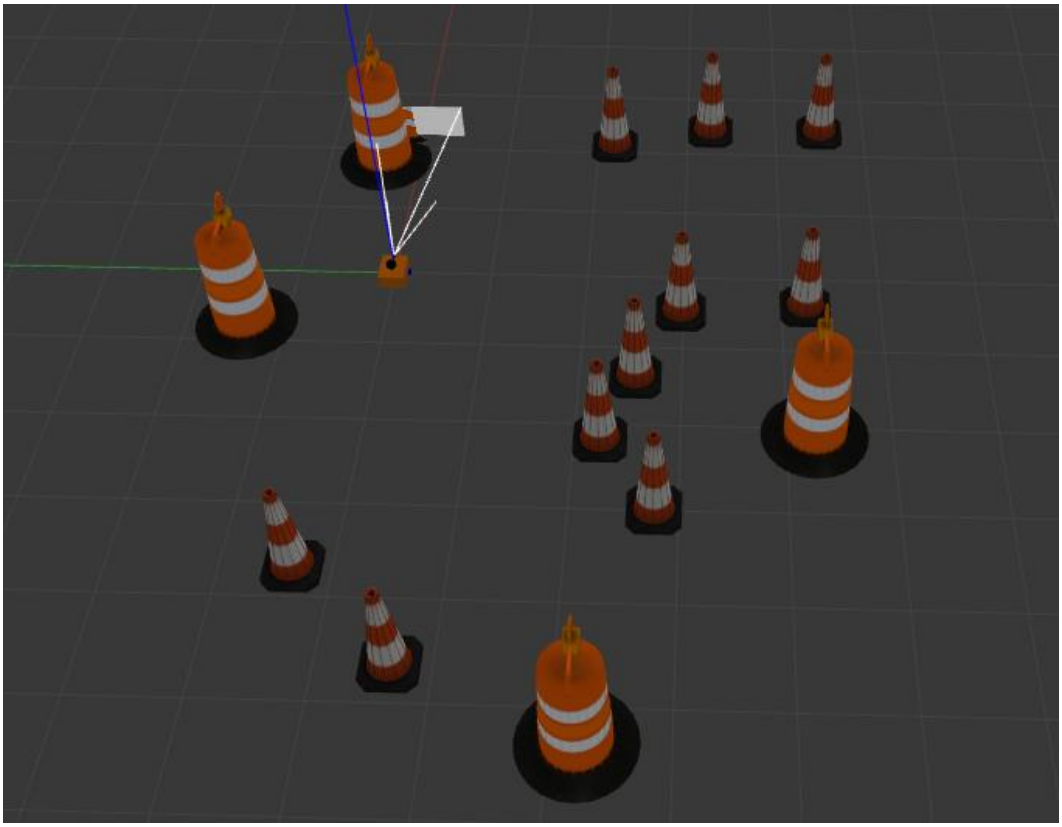

Figure 12: Rover deployed in the Gazebo Classic simulation environment with obstacle layout

The use of simulation also played a key role in debugging frame consistency, controller behavior, and sensor placement. Specifically, it allowed easier observing the way the rover was reacting to the environment and whether the software-side architecture was behaving as expected before transitioning to the hardware phase.

6.2 Rover Visualization and Sensor Representation

After configuring the simulation environment, the rover was visualized in **RViz** to ensure that the robot description, transforms and sensing outputs were correctly configured. This was a significant step since a proper visualization is usually the initial step towards ensuring the underlying ROS structure is coherent. When the robot is connected, sensor frames, or wheel behavior are not evenly distributed and these problems tend to manifest in RViz before they become apparent in other places.

The simulation of the LiDAR sensor was implemented using the standard ROS-Gazebo workflow, with scan data being published and displayed as the rover moved. This not only provided the rover with an operating perception layer in simulation, but also allowed it to test how information about obstacles was to be used later in the process, in navigation and costmap generation.

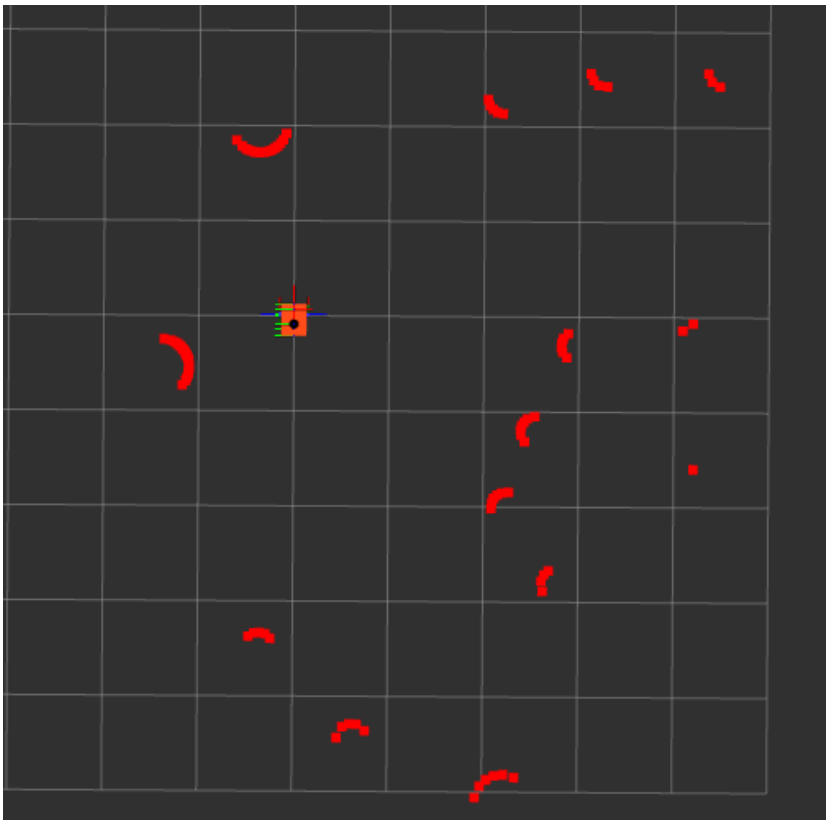

Figure 13: RViz visualization of the rover frame and LiDAR scan data

The visualization step also aided in verifying the relative position of the sensor frame and the rover body. Because LiDAR-based navigation requires the correct interpretation of the scan, it was necessary to ensure that the sensor was correctly aligned when simulating the navigation.

The RViz visualization ensured that the readings of the rover frame, LiDAR scan output, and surrounding obstacle readings were being represented properly in the ROS environment. The arcs of the red scan visible around the rover will show the detected boundaries of the obstacles using the LiDAR data. This was a significant step since it confirmed that the simulated sensor not only

existed in Gazebo but also published meaningful scan information to ROS and therefore can be visualized and used in later navigation.

6.3 SLAM and Navigation Workflow

After rover visualization and sensor validation, the next major stage was the integration of mapping and navigation. The simulated rover was used in a **SLAM-oriented workflow** to develop an understanding of the environment and to support later autonomous navigation tasks. This stage allowed the rover to move through the environment while incrementally building a map representation that could be used for localization and path planning.

The generated map in RViz shows how the rover used LiDAR observations to form an occupancy-based representation of the simulated environment. Obstacles placed in the Gazebo world appeared in the map as occupied regions, while the surrounding navigable area was represented as free or partially observed space. This confirmed that the SLAM workflow was able to convert raw scan data into a map suitable for navigation-related tasks.

The navigation workflow was built around standard ROS 2 principles in which perception, transforms, and command generation work together. LiDAR data provided environmental information, while the rover state and odometry supported map-relative movement. Once this data flow became stable, the rover could be used in a more complete autonomous navigation pipeline.

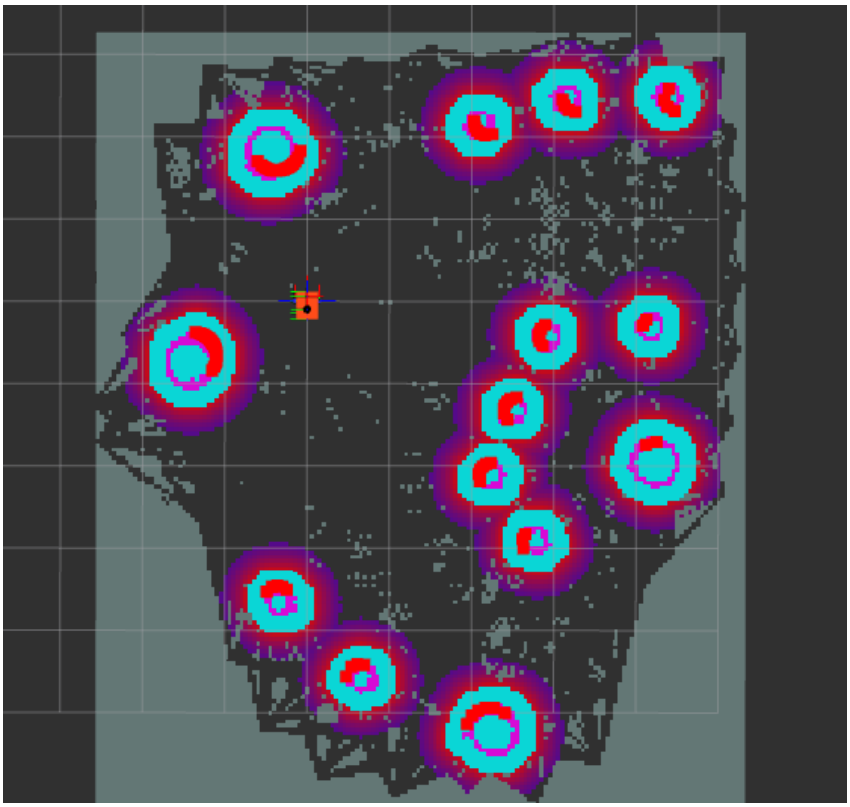

Figure 14: Generated SLAM map with obstacle representation in RViz

This phase was a milestone as it bridged the gap between low mobility at low levels and system autonomy at high levels. The rover could now be considered as a robot, which could be operated within a well-organized ROS navigation system.

Table 7: Major simulation and navigation functions used during rover deployment

Function / Module	**Role in the Project**
Gazebo Classic simulation	Provided the virtual environment for rover deployment and motion testing
RViz visualization	Enabled observation of robot model, LiDAR scans, maps, and navigation outputs
LiDAR sensor simulation	Generated scan data for obstacle perception and environment interaction
SLAM workflow	Supported map generation and environment understanding
Navigation stack	Enabled structured autonomous motion and path execution
Costmaps	Represented obstacle regions for safe navigation decisions
Detection-event logic	Simulated mine-related trigger behavior within the ROS system

6.4 Costmaps and Obstacle Handling

The use of **costmaps** to represent obstacles and ensure motion safety were an important part of the navigation workflow. In Nav2, costmaps enable the rover to view the environment not merely as raw LiDAR points, but as navigational areas where risk or cost can vary. This is necessary since not only should the robot be able to detect obstacles, but also ensure that during its motion, it is at a safe distance with the obstacles.

Global costmap is a representation of the mapped environment with a broader representation and is utilized in higher level path planning. It aids the navigation stack to be aware of the location of obstacles on the relative map of the whole. The **local costmap**, in turn, takes into account the immediate space around the rover and is updated more directly based on the observations of the nearby sensors. This local representation is significant in short distance obstacle avoidance and safe control generation.

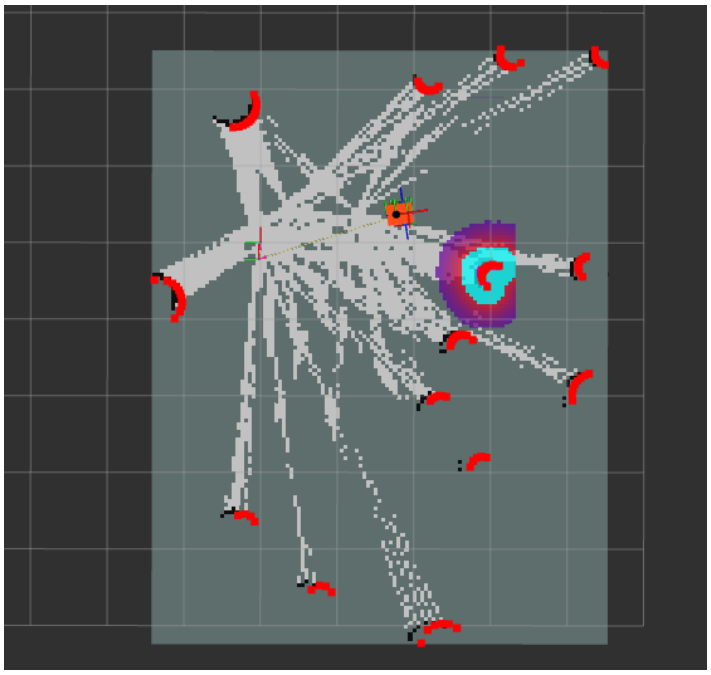

Figure 15: Local Costmap visualisation

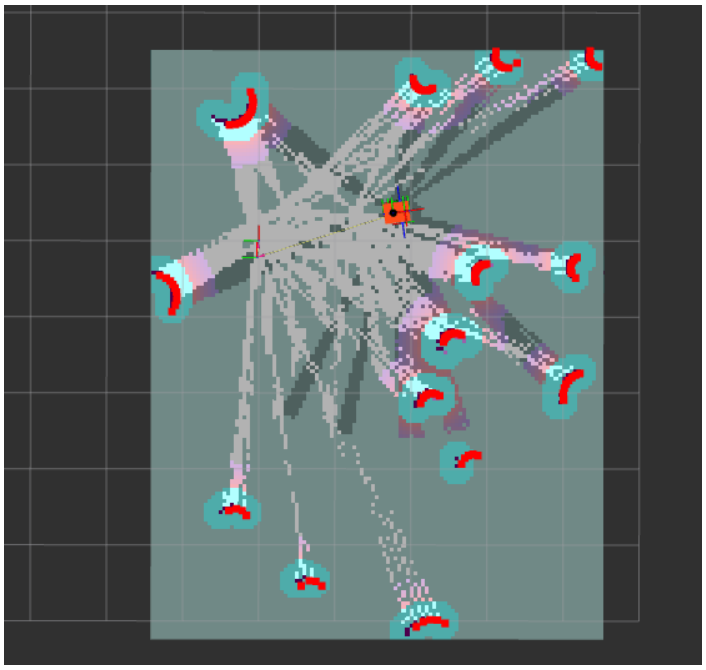

Figure 16: Global Costmap visualization

The RViz costmap screenshots depict the inflation of obstacles around the objects that are identified. The regions where there are inflation points are safety margins around obstacles, ensuring that the rover is not planning any path too near obstacles. This is of particular concern to a landmine-detection rover since the robot should move very slowly and only engage with the environmental obstacles when absolutely necessary. These visualizations ensured that the navigation stack was processing LiDAR-based obstacle information in a form that could be accessed and used.

6.5 Landmine Detection Workflow in the ROS Environment

Besides navigation, a landmine-detection-oriented logic layer was also considered to be part of the simulated project workflow. This layer was meant to model the behavior of the rover to suspected mine locations in the larger ROS system. Instead of viewing the rover as a purely navigation platform, this workflow linked the motion of the rover with mission-specific detection behavior.

They might be considered as hidden target areas or, event-trigger areas, in the simulation-based interpretation of the system. When the rover entered or approached such a region the system could generate a detection-related response, which could then be published, visualized or logged within the ROS environment. This further brought the simulation to a closer resemblance of the real application environment of the project.

This workflow also contributed to developing continuity between simulation and real implementation. The mine-related events were conceptually manipulated in simulation using ROS-based logic. This same role was later enabled in hardware by the DIY metal detector and

Arduino-to-ROS communication path. Due to this, the simulation stage did not stand on its own in comparison to the actual rover; it was really in support of the subsequent sensing architecture.

6.6 Summary

This chapter described how the rover was developed and validated in simulation before moving to the physical platform. The rover was integrated into **Gazebo Classic** and visualized through **RViz**, allowing the robot model, LiDAR sensing, and ROS data flow to be verified. The chapter also outlined the SLAM and navigation workflow, the role of costmaps in obstacle handling, and the way landmine-detection-oriented logic was represented within the ROS environment.

Together, these simulation components formed the foundation for the later physical implementation phase. They allowed the rover's navigation behavior, sensor structure, and system-level workflow to be tested in a repeatable environment before being transferred to real hardware.

Chapter 7: Physical Implementation, Testing, Results, and Discussion

7.1 Physical Rover Implementation

After validating the rover model and navigation workflow in simulation, the system was implemented as a physical rover. The purpose of this stage was to transfer the simulation-based concept onto real hardware and verify that the rover could operate as a functional ROS-based mobile robot.

The physical rover consisted of a **Raspberry Pi 4 with 8 GB RAM**, **Arduino Nano**, **L298N motor driver**, **76 RPM encoder motors**, **RPLidar A1**, **DIY metal detector**, **3S 11.1 V LiPo battery**, and a separate **power bank for the Raspberry Pi**. The Raspberry Pi acted as the main onboard computer running ROS, while the Arduino Nano handled low-level motor and sensor interfacing through the ROS Arduino Bridge.

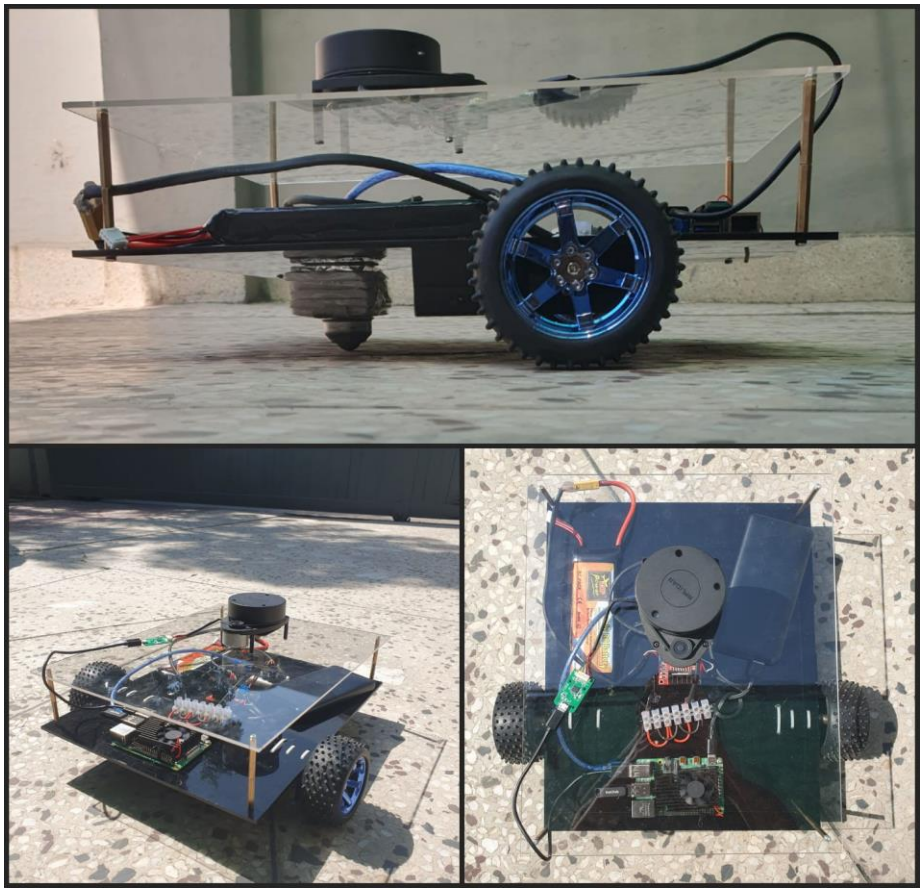

Figure 17: Physical rover prototype developed after simulation validation

The final rover employed a **two wheel differential-drive design with caster**. This design was chosen following the previous four-wheel design which generated problems of odometry drift and wheel slide. The differential-drive configuration made the motion model easier to understand and enhanced its compatibility with the ROS control and navigation workflow.

Figure 17 displays the major hardware components that will be used in the rover. These elements were chosen to make the system low-cost, modular, and compatible with the development based on ROS. The Raspberry Pi served as the onboard ROS computer, the Arduino Nano was used to interfere with the lower levels, the L298N was used to drive the motors, the RPLidar A1 was used to provide the data of the environmental scan, and the DIY metal detector was used to provide the analog signal of detection of a suspected metallic object.

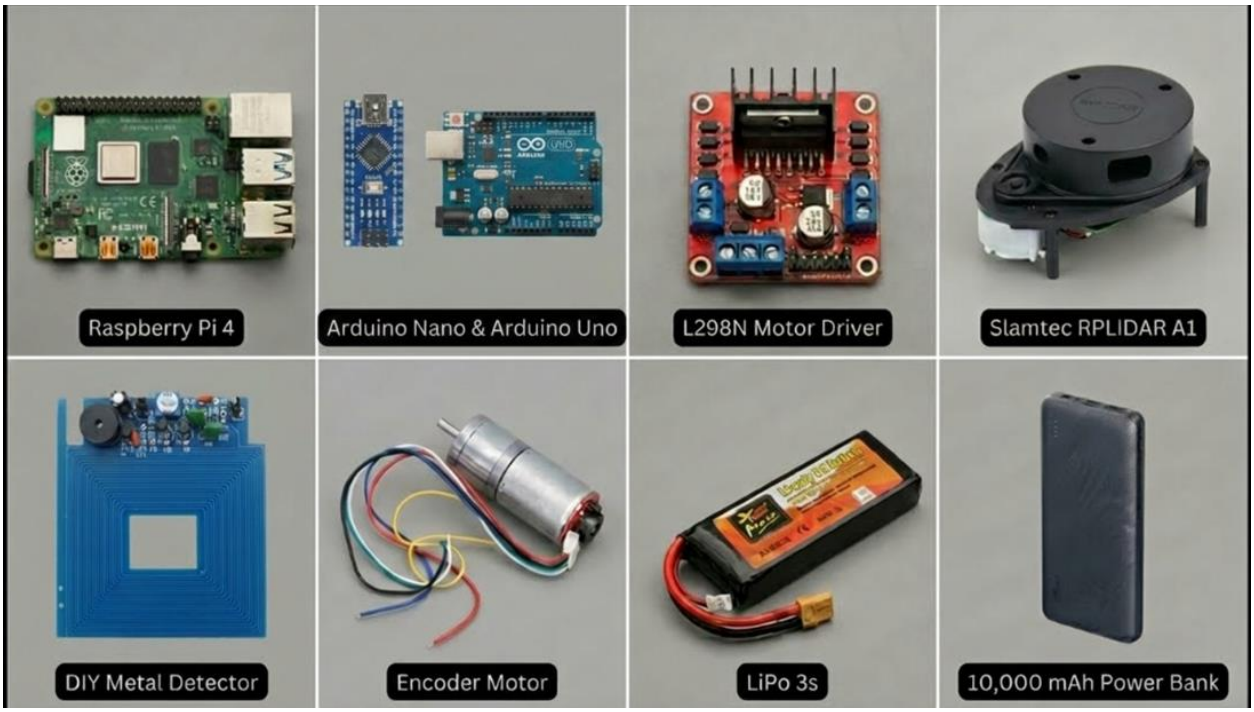

Figure 18: Major Hardware components integrated on the physical rover

Table 8: Physical Rover implementation checklist

Subsystem	**Implemented Component**	**Purpose**
Main computing	Raspberry Pi 4, 8 GB	Runs ROS and high-level rover software
Low-level interface	Arduino Nano	Interfaces motor control and metal detector signal
Motor driver	L298N	Drives the left and right motors
Drive system	76 RPM encoder motors	Provides rover movement
Navigation sensor	RPLidar A1	Provides LiDAR scan data
Detection sensor	DIY metal detector	Detects nearby metallic objects through analog voltage change
Main power source	3S 11.1 V LiPo battery	Powers rover actuation system
Pi power source	Power bank	Provides stable power to Raspberry Pi
Chassis	Two-wheel rover with caster	Provides mechanical structure and mobility

7.2 Hardware Interfacing and Motor Control

The motor control system was separated into two boards: the Raspberry Pi and the Arduino Nano. The Raspberry Pi was used to provide the ROS-side communication and higher-level system logic, and the Arduino Nano was used to provide the low-level interface between ROS and the motor driver. This helped to make the system more modular and easier to debug.

The **ROS Arduino Bridge** was employed to get the ROS environment on the Raspberry Pi to interface with the Arduino Nano. This bridge received motion commands via ROS and interpreted them via the Arduino and then decoded them into control signals sent to the **L298N motor driver**. The L298N subsequently powered the left and right 76 RPM encoder motors.

This set-up enabled the rover to have a clear command flow between ROS and physical movement:

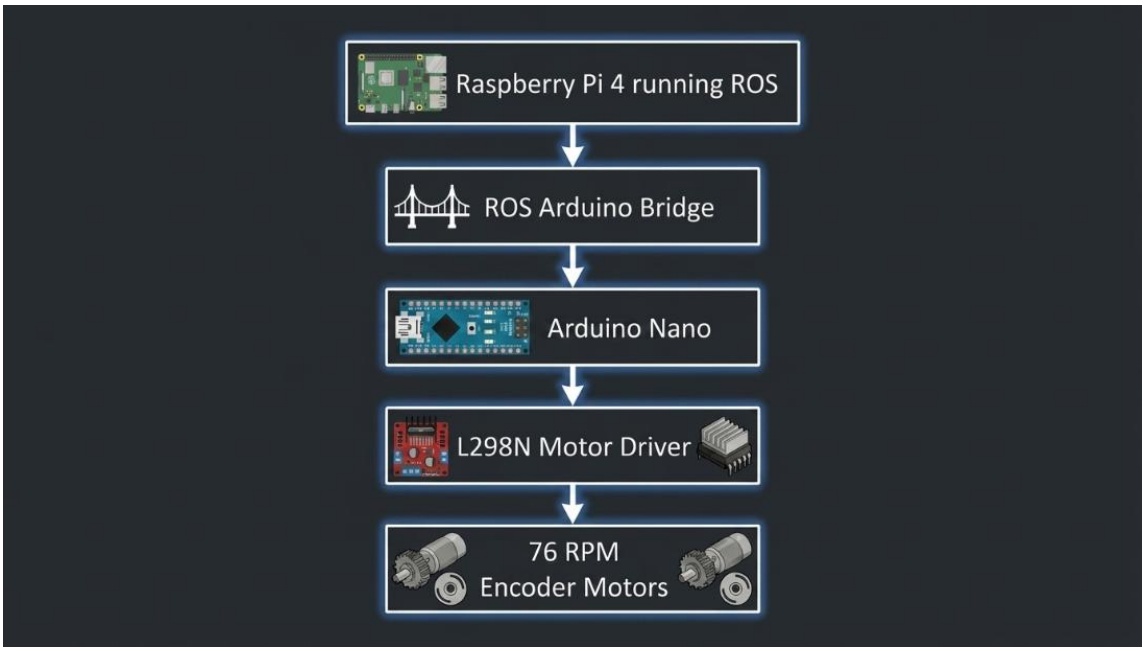

Figure 19: Hardware motor control chain from ROS to physical rover motion

This layered control approach also made the hardware implementation closer to the simulation architecture, where ROS commands were already being used to represent rover motion.

7.3 Metal Detector Integration

The physical landmine detector on the physical rover was implemented using a **DIY metal detector**. The detector works on the principle that the presence of nearby metal varies the magnetic field around the detector coil. This change influences the magnetic flux and results in a similar change in the detector output voltage.

The **analog input of the Arduino UNO** was connected to the output of the metal detector. The Arduino responded to this changing voltage and compared it to a threshold to determine whether a detection event had taken place. After detecting a metallic object, the Arduino would send the event to the ROS system running on the Raspberry Pi.

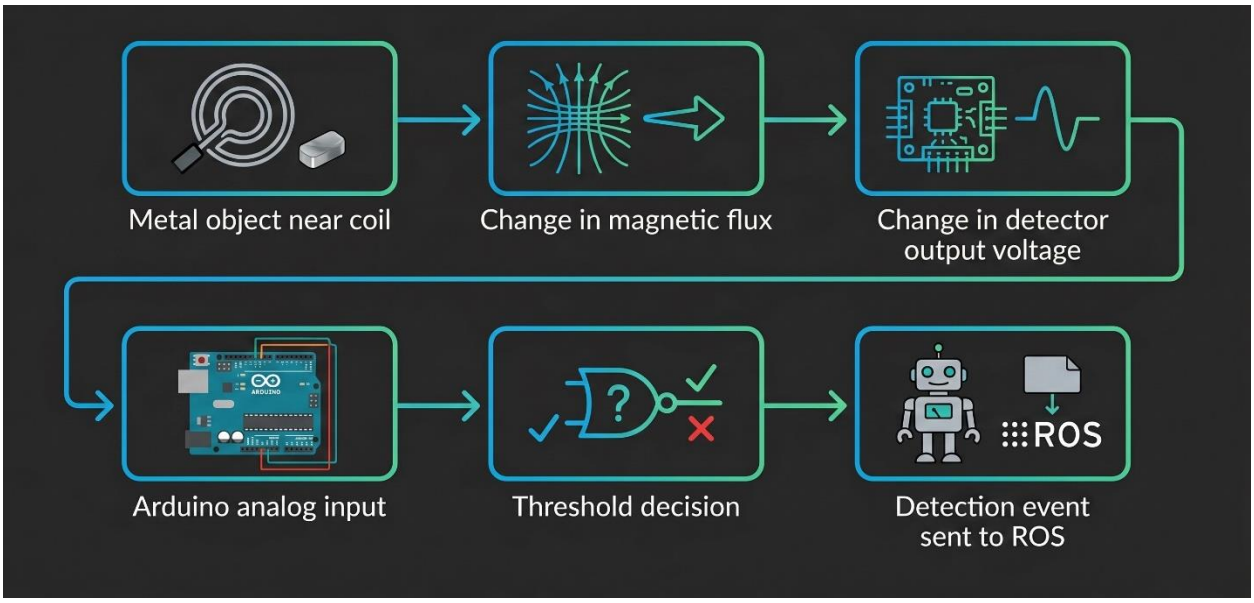

Figure 20: Signal Flow of the DIY metal detector integrated with the ROS-based rover

This design enabled the detector to be integrated into the ROS-based workflow of the rover, as opposed to being a stand-alone circuit. This detection event may subsequently be used to put the rover into stop, place a mark where the object has been detected, publish an alert, or generate a visualization marker in RViz.

7.4 ROS System Validation

Prior to final testing, the ROS system was also tested to make sure that the necessary frames, transforms, and communication paths were functional. The reason behind this step was that navigation, SLAM, LiDAR visualization, and odometry all rely on a uniform frame structure in ROS.

The ROS frame visualization tools were used to check the TF tree. The combination of the TF tree generated illustrated the connection between key frames like **map, odom, base-link, base-footprint, laser-frame, wheel frames, chassis** and **caster wheel**. The existence of these frames validated that the rover model was structurally interconnected in the ROS environment. There is also active publication of transforms on frames (odom and map) which are fundamental to localization and navigation behavior.

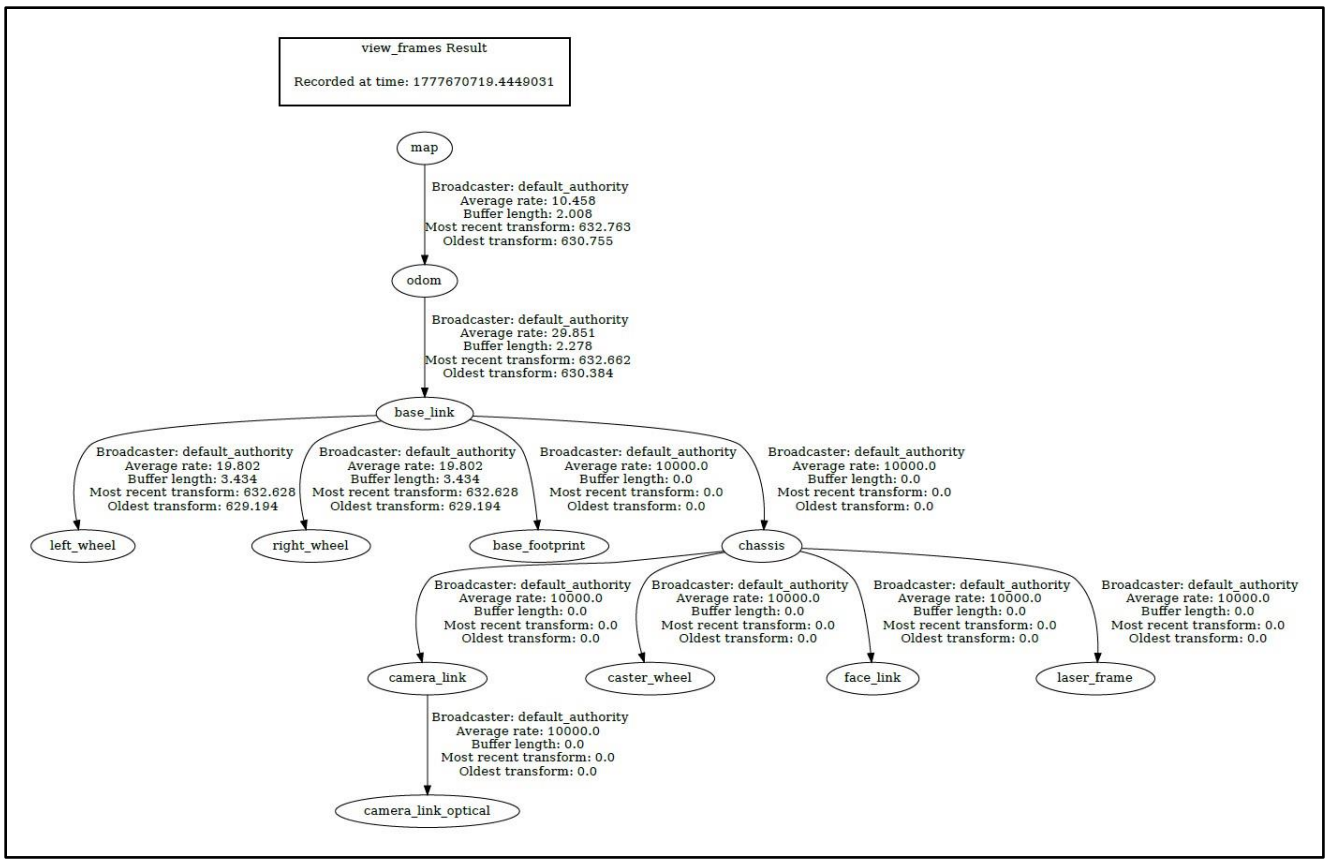

Figure 21: TF Tree verification of the rover coordinate frame structure

This test was done to ensure that the rover had a functional coordinate frame system. Incorrect TF tree can cause LiDAR scans to appear in the incorrect location, costmaps to not align with the robot, and Nav2 to not generate feasible paths.

7.5 Testing Procedure

The testing was done in phases to minimize the complexity of debugging. Rather than testing the entire rover simultaneously, each of the large subsystems was initially checked on its own, and then was also tested as a part of the overall system.

The initial step was to examine the motor control chain. Raspberry Pi, ROS Arduino Bridge, Arduino Nano, L298N motor driver, and motors were tested to ensure that commands of motion led to the appropriate movement of the wheels. The second phase consisted in testing RPLidar A1 to make sure that scan data was being received and visualized correctly in ROS. The third step was to test the DIY metal detector by placing metallic objects close to the detector coil and measureing the analog voltage response over the Arduino.

Figure 22: Physical Testing Setup used for rover navigation and detection validation

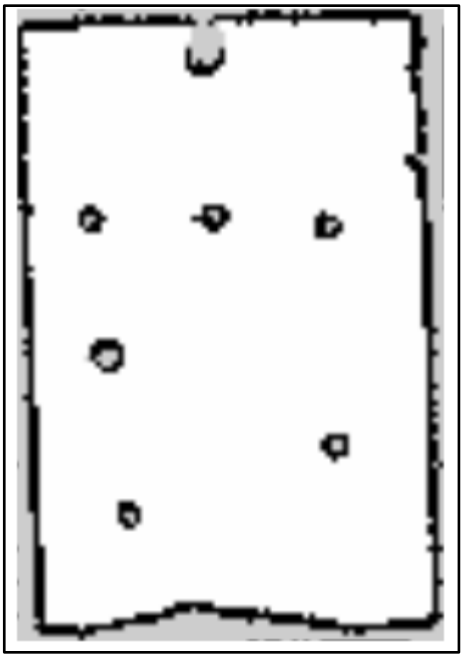

Figure 23: SLAM Map generated of the Physical Environment

After subsystem verification, integrated testing was performed. During this step, the rover was powered on as a complete system, ROS nodes were launched, LiDAR data was monitored, motor commands were sent, and the detector response was checked. This was used to validate that the rover could be used as a combined sensing, control and detection platform.

Table 9: Testing and Validation summary of the physical rover

Test Area	Test Performed	Expected Outcome	Observed Outcome
Motor control	Sent motion commands from ROS through Arduino bridge	Motors respond correctly to movement commands	**Passed.** Motors responded to forward, reverse, left, right, and stop commands sent through the ROS-Arduino control chain.
ROS Arduino Bridge	Checked communication between Raspberry Pi and Arduino Nano	Stable serial communication between ROS and Arduino	**Passed.** Raspberry Pi and Arduino Nano communicated successfully over serial, allowing ROS-side commands to reach the motor-control layer.
LiDAR sensing	Launched RPLidar A1 and visualized scan data	LiDAR scan visible in ROS/RViz	**Passed.** RPLidar A1 scan data was successfully published and visualized in RViz. Obstacle readings were visible around the rover.
TF validation	Inspected TF tree using ROS tools	Correct frame relationship between map, odom, base_link, and sensors	**Passed.** TF tree showed valid frame relationships between **map**, **odom**, **base_link**, wheel frames, and **laser_frame**.

Metal detection	Placed metallic object near detector coil	Analog voltage change detected by Arduino	**Passed.** Metallic objects near the detector coil produced a voltage change that was read by the Arduino analog input.
Integrated rover operation	Ran rover with sensing and control subsystems active	Rover operates as combined ROS-based system	**Partially Passed.** The rover operated as a combined ROS-based system with motor control, LiDAR sensing, and metal detection active. Further tuning is required for more reliable autonomous field operation.

7.6 Results and Discussion

The physical rover was able to achieve a demonstration of the simulation-hardware implementation transition. The rover was able to use the **Raspberry Pi** as the main ROS computer, communicate with the Arduino Nano through the ROS Arduino Bridge, and actuate the motors through the L298N motor driver. This ensured that the simple ROS-to-hardware command chain worked.

The **RPLidar A1** was a perception input required by the rover, as well as supporting the same LiDAR-based concept that had previously been proven in simulation. LiDAR usage was beneficial in ensuring the flow of the navigation work in the simulator and the actual implementation of the rover.

The **DIY metal detector** also gave a response to other metallic objects around using the analog voltage variation. This confirmed the possibility of using the Arduino UNO as an interface between the detector circuit and the ROS system. Even though the detector in this project is a low-cost DIY implementation, it provides a valuable demonstration of concept to mine-detection-oriented rover behavior.

The significant design lesson of the project was that the mechanical configuration of the rover should be matched with the assumptions of the software control system. The previous four-wheel design seemed to be mechanically stable but led to slippage of wheels and inconsistency of odometry. It turned out that the last two-wheel differential-drive system was more appropriate to the ROS-based navigation and control.

Overall, the results show that the system achieved its main goal: a simulation-validated rover concept was successfully implemented as a physical ROS-based rover with LiDAR sensing, motor control, and metal detection capability.

Table 10: Main observations and engineering lessons from implementation

Observation	**Discussion / Lesson Learned**
Four-wheel configuration caused odometry issues	The initial design introduced wheel slippage and odometry shift, making navigation less reliable.
Two-wheel differential drive improved behavior	The final design better matched ROS differential-drive assumptions and simplified motion control.
Simulation reduced hardware debugging effort	Gazebo and RViz allowed testing of model, LiDAR, TF, and navigation workflow before hardware implementation.
Separate Pi and Arduino roles improved structure	Raspberry Pi handled ROS, while Arduino handled low-level interfacing, making the system easier to manage.
Metal detector integration was feasible	Analog voltage variation from the detector could be interpreted by Arduino and connected to ROS-side logic.
Sensor placement matters	LiDAR needed clear visibility for scans, while the metal detector required low mounting near the ground.

7.7 Summary

This chapter detailed the actual physical implementation, hardware interfacing, ROS validation, testing process, and overall results of the rover. The final system consisted of a Raspberry Pi 4, Arduino Nano, Arduino UNO, ROS Arduino Bridge, L298N motor driver, 76 RPM encoder motors, RPLidar A1, DIY metal detector, LiPo battery, and power bank into a working rover platform.

Also discussed in the chapter was how the rover was tested in phases and how the TF tree was employed to validate the ROS frame structure. These findings confirmed that the project had been transitioning to real hardware without breaking the principle main ROS-based workflow developed earlier.

Chapter 8: Conclusion and Future Work

8.1 Conclusion

This project involved the design, simulation and physical engagement of a **LiDAR Guided Rover to detect Landmines**. The overall objective of the project was to come up with a rover-based system that could be used to inspect the hazardous areas and as well lower direct human exposure in the initial stages of landmine survey. The project merged independent mobile robotics, LiDAR based perception, ROS based software integration, simulated validation and physical hardware implementation into one operational platform.

Simulation was done using **ROS 2 Humble** and **Gazebo Classic** These models were tested with simulation before hardware implementation. The simulation-first methodology was handy as it permitted the detection of the problems in robot behaviour, sensor placement, and navigation logic earlier than the actual construction of the physical rover. The visualization of the rover model, LiDAR scan data, generated maps, costmaps, and the TF frame structure were all extensively visualized using RViz.

Raspberry Pi 4, **Arduino Nano, Arduino UNO, ROS Arduino Bridge, L298N motor driver, 76 RPM encoder motors, RPLidar A1, DIY metal detector, 3S 11.1 V LiPo battery** and a separate **power bank** to the Raspberry Pi were used to implement the physical rover. The Raspberry Pi was the central ROS computer, and the Arduino Nano was the low-level motor interface and analog detector input device of the metal detector. Such separation of duties rendered the system modular and simpler to debug.

One significant engineering deliverable of the project was the change in design of a **four-wheel rover design to a two-wheel differential-drive design with caster support**. The four-wheel design brought about wheel slippage, odometry shift and this influenced the stability of the navigation. The final two-wheel differentiating design offered more predictable movement and enhanced compliance with ROS-founded control and movement presumptions.

Overall, the project was able to illustrate a whole robotics workflow, including simulation and real implementation. It was possible to integrate LiDAR sensing, ROS-based control, motor actuation, validation of the TF, and metal-detection-oriented sensing into one integrated system. Though the prototype is not a field-ready autonomous hazardous-environment survey robot, it provides a very solid basis on which future development of autonomous hazardous-environment survey robots will be developed.

8.2 Project Limitations

Even though the project has fulfilled its primary goals, there are several limitations too. The first constraint is that the rover was designed and tested more as a prototype than as a rugged field-ready system. Actual minefield conditions involve unequal ground, ground variability, vegetation, dust, moisture and unpredictable obstacles. In its current form, the rover platform would need to

be mechanically reinforced and environmentally protected prior to being deployed into the outdoor field environment.

The other constraint is the detection mechanism. The **DIY metal detector** could react to the presence of metallic objects by changing the voltage in an analog circuit, but it is, nonetheless, a cheap DIY project. The range of its detection, sensitivity and false-positive behaviour would require additional calibration and controlled experiments before it could be deemed suitable to practical mine survey work. In practice applications of demining, frequently metal detection itself is not adequate since metallic objects may also produce a detection.

The navigation system needs also additional refinement of the system to be physically deployed. Although the results of the simulation indicated the successful LiDAR visualization, mapping, costmap generation, and navigation-related behaviour, the real-world operation presents further uncertainty. The causes of odometry and localization error can include wheel slippage, surface irregularities, sensor noise, and power variations.

The existing implementation lacks the GPS-based logging of coordinates as well and a full coverage path planner of the minefield. In practice, to survey a real mine, the rover would have to record the suspected mine locations in an accurate manner and follow a systematic coverage pattern to ensure that no part of the test area is overlooked.

8.3 Future Work

The project can be enhanced in various ways in future work. The initial significant enhancement would be the introduction of **GPS** location tagging. This would enable the rover to log the rough position of any perception of a mine and produce a map of dangerous sites which could be examined later by humans.

The second thing that could be improved is the introduction of a structured coverage path planning algorithm. This would ensure the rover is more appropriate in systematic inspection of the area rather than situational general navigation behaviour only.

It is also possible to enhance the sensing system. A more sophisticated metal detector like a pulse-induction-based detector might be more effective in increasing the detection depth and decreasing false alarms. The rover can also be equipped with **gas sensors** to detect any dangerous vapors or traces of chemicals and a **camera** can be added to provide real-time visual observations. The camera would enable an operator to visually inspect the rover environment, and gas sensors would make the platform more practical to more applications involving hazardous environments.

Mechanical design could be also enhanced with a more rugged chassis, better wheel traction, weather-resistant enclosures, and a better mounting of the LiDAR and detector coil. Such modifications would render the rover more appropriate to test outdoor terrains.

Lastly, the project can be further extended to the multi-robot operation. Larger areas could be covered more swiftly by a group of similar rovers, share map data and divide survey regions among themselves. This would provide a more scalable system to large scale inspection work over a large area.

Table 11: Summary of Proposed Future Enhancements

Future Enhancement	Purpose / Expected Benefit
GPS integration	Allows suspected mine locations to be logged with coordinates for later review
Coverage path planning	Enables systematic area scanning and reduces the chance of missing sections
Improved metal detector	Increases detection reliability, depth range, and noise immunity
Camera module	Provides real-time visual feedback for operator monitoring
Gas sensors	Expands the rover into a broader hazardous-environment sensing platform
Rugged chassis design	Improves outdoor usability and mechanical durability
Better wheel traction	Reduces slippage and improves odometry reliability
Sensor fusion	Combines LiDAR, encoder, detector, camera, and other sensor data for stronger decision-making

8.4 Final Remarks

The **LiDAR Guided Rover for Landmine Detection** illustrates how modern robotics tools could be utilized to come up with a viable and meaningful engineering prototype. With the integration of ROS 2, Gazebo simulation, LiDAR sensing, differential-drive control, Arduino based motor interfacing and a DIY metal detector, the project demonstrates a whole pathway of the idea to the working rover platform.

Another key engineering lesson that the project demonstrates is that the development of successful robotics requires mechanical design, sensing, software architecture, and control assumptions to be in harmony. This change of the original four-wheel design to the final differential-drive design demonstrated how a simpler design could in many cases be used to achieve more reliable robotic behaviour.

Although it would be premature to consider such a rover would be useful in the real demining operations, the system developed in this project would give a solid base in future research and development. The platform can be expanded to more realistic hazardous-area inspection and humanitarian robotics applications with better sensing, better localization, GPS logging, coverage planning, and rugged hardware.

Chapter 9: References

[1] International Campaign to Ban Landmines – Cluster Munition Coalition, **Landmine Monitor 2023**, Geneva, Switzerland, 2023.

[2] Landmine and Cluster Munition Monitor, **Landmine Monitor Reports**, 2025.

[3] Open Robotics, **ROS 2 Humble Hawksbill Documentation**, ROS 2 Documentation.

[4] Open Robotics, **Gazebo Tutorials for ROS 2 Humble**, ROS 2 Documentation.

[5] Gazebo, **ROS 2 Integration Overview for Gazebo Classic**, Gazebo Classic Documentation.

[6] Open Navigation LLC, **Nav2 Documentation: ROS 2 Navigation Stack**, Nav2 Documentation.

[7] Open Navigation LLC, **Nav2 Navigation System Overview**, Nav2 Project Website.

[8] SLAMTEC Co., Ltd., **RPLIDAR A1 Development Kit User Manual**, Model A1M8, Rev. 01, 2016.

[9] Raspberry Pi Ltd., **Raspberry Pi 4 Model B Specifications**, Raspberry Pi Documentation.

[10] Arduino, **Arduino Nano Rev3 Documentation**, Arduino Documentation.

[11] STMicroelectronics, **L298 Dual Full-Bridge Driver Datasheet**, STMicroelectronics, 2023.

[12] J. Newans, **Articubot One Robot Package Template**, GitHub Repository.

[13] J. Newans, **Articulated Robotics: Mobile Robot Project Overview**, Articulated Robotics.

[14] R. R. Murphy, **Disaster Robotics**, Cambridge, MA, USA: MIT Press, 2014.

[15] S. Thrun, W. Burgard, and D. Fox, **Probabilistic Robotics**, Cambridge, MA, USA: MIT Press, 2005.

APPENDIX A: IMPORTANT CODE AND CONFIGURATION FILES

A.1 Main Robot Description File

This file represents the main Xacro entry point of the rover model. It includes the core robot body, ROS 2 control configuration, LiDAR module, camera module, and other robot components. It also allows switching between real robot control and simulation mode.

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="robot">

    <xacro:arg name="use_ros2_control" default="true"/>
    <xacro:arg name="sim_mode" default="false"/>

    <xacro:include filename="robot_core.xacro" />

    <xacro:if value="$(arg use_ros2_control)">
        <xacro:include filename="ros2_control.xacro" />
    </xacro:if>
    <xacro:unless value="$(arg use_ros2_control)">
        <xacro:include filename="gazebo_control.xacro" />
    </xacro:unless>

    <xacro:include filename="lidar.xacro" />
    <xacro:include filename="camera.xacro" />
    <xacro:include filename="face.xacro" />

</robot>
```

A.2 Rover Core Geometry and Wheel Configuration

The following snippet defines the main dimensional parameters of the rover, including chassis size, wheel radius, wheel offsets, and caster wheel properties. These parameters are important because they directly affect the simulated rover geometry, wheel placement, and differential-drive behavior.

```
<xacro:property name="chassis_length" value="0.335"/>
<xacro:property name="chassis_width" value="0.265"/>
<xacro:property name="chassis_height" value="0.138"/>
<xacro:property name="chassis_mass" value="1.0"/>

<xacro:property name="wheel_radius" value="0.033"/>
<xacro:property name="wheel_thickness" value="0.026"/>
<xacro:property name="wheel_mass" value="0.05"/>
<xacro:property name="wheel_offset_x" value="0.226"/>
<xacro:property name="wheel_offset_y" value="0.1485"/>
<xacro:property name="wheel_offset_z" value="0.01"/>

<xacro:property name="caster_wheel_radius" value="0.01"/>
<xacro:property name="caster_wheel_mass" value="0.01"/>
<xacro:property name="caster_wheel_offset_x" value="0.075"/>
```

The left and right wheel joints are defined as continuous joints, allowing the wheels to rotate during differential-drive movement.

```
<joint name="left_wheel_joint" type="continuous">
    <parent link="base_link"/>
    <child link="left_wheel"/>
    <origin xyz="0 ${wheel_offset_y} 0" rpy="-${pi/2} 0 0" />
    <axis xyz="0 0 1"/>
</joint>
<joint name="right_wheel_joint" type="continuous">
    <parent link="base_link"/>
    <child link="right_wheel"/>
    <origin xyz="0 ${-wheel_offset_y} 0" rpy="${pi/2} 0 0" />
    <axis xyz="0 0 -1"/>
</joint>
```

A.3 LiDAR Sensor Xacro Configuration

The LiDAR module is attached to the chassis using a fixed joint. The laser_frame is important because LiDAR scan data must be published relative to a known robot frame for SLAM, mapping, costmaps, and navigation.

```
<joint name="laser_joint" type="fixed">
    <parent link="chassis"/>
    <child link="laser_frame"/>
    <origin xyz="0.122 0 0.212" rpy="0 0 0"/>
</joint>

<link name="laser_frame">
    <visual>
        <geometry>
            <cylinder radius="0.05" length="0.04"/>
        </geometry>
        <material name="black"/>
    </visual>

    <collision>
        <geometry>
            <cylinder radius="0.05" length="0.04"/>
        </geometry>
    </collision>
</link>
```

The Gazebo LiDAR plugin publishes simulated laser scan data as a **sensor_msgs/LaserScan** message from the **laser_frame**.

```
<gazebo reference="laser_frame">
    <material>Gazebo/Black</material>

    <sensor name="laser" type="ray">
        <pose>0 0 0 0 0 0</pose>
        <visualize>false</visualize>
        <update_rate>10</update_rate>
```

```
        <ray>
            <scan>
                <horizontal>
                    <samples>360</samples>
                    <min_angle>-3.14</min_angle>
                    <max_angle>3.14</max_angle>
                </horizontal>
            </scan>
            <range>
                <min>0.3</min>
                <max>12</max>
            </range>
        </ray>

        <plugin name="laser_controller" filename="libgazebo_ros_ray_sensor.so">
            <ros>
                <argument>~/out:=scan</argument>
            </ros>
            <output_type>sensor_msgs/LaserScan</output_type>
            <frame_name>laser_frame</frame_name>
        </plugin>
    </sensor>
</gazebo>
```

A.4 ROS2 Control and Arduino Bridge Configuration

This configuration connects the real robot hardware with ROS 2 control. In real robot mode, the system uses the **diffdrive_arduino** hardware plugin to communicate with the Arduino-based motor interface. The left and right wheel joints are controlled using velocity command interfaces.

```
<xacro:unless value="$(arg sim_mode)">
    <ros2_control name="RealRobot" type="system">
        <hardware>
            <plugin>diffdrive_arduino/DiffDriveArduino</plugin>
            <param name="left_wheel_name">left_wheel_joint</param>
```

```
            <param name="right_wheel_name">right_wheel_joint</param>
            <param name="loop_rate">30</param>
            <param name="device">/dev/ttyUSB0</param>
            <param name="baud_rate">57600</param>
            <param name="timeout">1000</param>
            <param name="enc_counts_per_rev">3436</param>
        </hardware>

        <joint name="left_wheel_joint">
            <command_interface name="velocity">
                <param name="min">-10</param>
                <param name="max">10</param>
            </command_interface>
            <state_interface name="velocity"/>
            <state_interface name="position"/>
        </joint>

        <joint name="right_wheel_joint">
            <command_interface name="velocity">
                <param name="min">-10</param>
                <param name="max">10</param>
            </command_interface>
            <state_interface name="velocity"/>
            <state_interface name="position"/>
        </joint>
    </ros2_control>
</xacro:unless>
```

For simulation mode, the same wheel joints are connected to the Gazebo ROS 2 control system.

```
<xacro:if value="$(arg sim_mode)">
    <ros2_control name="GazeboSystem" type="system">
        <hardware>
            <plugin>gazebo_ros2_control/GazeboSystem</plugin>
        </hardware>
```

```
        <joint name="left_wheel_joint">
            <command_interface name="velocity">
                <param name="min">-10</param>
                <param name="max">10</param>
            </command_interface>
            <state_interface name="velocity"/>
            <state_interface name="position"/>
        </joint>

        <joint name="right_wheel_joint">
            <command_interface name="velocity">
                <param name="min">-10</param>
                <param name="max">10</param>
            </command_interface>
            <state_interface name="velocity"/>
            <state_interface name="position"/>
        </joint>
    </ros2_control>
</xacro:if>
```

A.5 Differential Drive Controller Configuration

The differential-drive controller defines the wheel joints, wheel separation, wheel radius, publish rate, and command velocity interface. These values are important for generating odometry and converting velocity commands into wheel motion.

```
controller_manager:
  ros__parameters:
    update_rate: 30

    diff_cont:
      type: diff_drive_controller/DiffDriveController

    joint_broad:
```

```
      type: joint_state_broadcaster/JointStateBroadcaster


diff_cont:
  ros__parameters:


    publish_rate: 50.0
    base_frame_id: base_link


    left_wheel_names: ['left_wheel_joint']
    right_wheel_names: ['right_wheel_joint']


    wheel_separation: 0.297
    wheel_radius: 0.033


    use_stamped_vel: false
```

A.6 Main Robot Launch File

The main robot launch file starts the robot state publisher, twist multiplexer, controller manager, differential-drive controller, and joint state broadcaster. It also remaps the output command velocity topic toward the differential-drive controller. Your uploaded launch file includes the robot state publisher, twist mux, controller manager, and delayed controller spawners.

```
package_name = 'articubot_one'


rsp = IncludeLaunchDescription(
    PythonLaunchDescriptionSource([os.path.join(
        get_package_share_directory(package_name), 'launch', 'rsp.launch.py'
    )]),
    launch_arguments={
        'use_sim_time': 'false',
        'use_ros2_control': 'true'
    }.items()
)


twist_mux_params = os.path.join(
```

```
    get_package_share_directory(package_name),
    'config',
    'twist_mux.yaml'
)

twist_mux = Node(
    package="twist_mux",
    executable="twist_mux",
    parameters=[twist_mux_params],
    remappings=[('/cmd_vel_out', '/diff_cont/cmd_vel_unstamped')]
)

controller_manager = Node(
    package="controller_manager",
    executable="ros2_control_node",
    parameters=[{'robot_description': robot_description},
                controller_params_file]
)
diff_drive_spawner = Node(
    package="controller_manager",
    executable="spawner.py",
    arguments=["diff_cont"],
)
joint_broad_spawner = Node(
    package="controller_manager",
    executable="spawner.py",
    arguments=["joint_broad"],
)
```

A.7 SLAM Toolbox Parameters

```
slam_toolbox:
  ros__parameters:

    odom_frame: odom
    map_frame: map
    base_frame: base_footprint
    scan_topic: /scan
    mode: localization

    map_file_name: /home/dev/dev_ws/my_map_serial
    map_start_at_dock: true

    transform_publish_period: 0.02
    map_update_interval: 5.0
    resolution: 0.05
    max_laser_range: 20.0
    transform_timeout: 0.2
    tf_buffer_duration: 30.
    enable_interactive_mode: true

    use_scan_matching: true
    do_loop_closing: true
```

A.8 Nav2 Controller

```
controller_server:
  ros__parameters:
    use_sim_time: True
    controller_frequency: 20.0
    progress_checker_plugin: "progress_checker"
    goal_checker_plugin: "goal_checker"
    controller_plugins: ["FollowPath"]
```

```
    FollowPath:
      plugin: "dwb_core::DWBLocalPlanner"
      max_vel_x: 0.26
      max_vel_theta: 1.0
      acc_lim_x: 2.5
      acc_lim_theta: 3.2
      sim_time: 1.7
      critics: ["RotateToGoal", "Oscillation", "BaseObstacle",
                "GoalAlign", "PathAlign", "PathDist", "GoalDist"]
```

A.9 Arduino Code for Metal Detection and Buzzer Alert

The Arduino code below reads the analog output of the DIY metal detector. The detector output voltage exceeds a predetermined threshold when a metallic object is detected. Arduino turns on a buzzer and transmits a detection message through the serial communication to the Raspberry Pi. This event can then be used by the ROS-side system to temporarily interrupt the rover, reverse, move about the detected position and proceed with navigation to the active Nav2 goal.

```
const int metalSensorPin = A0;
const int buzzerPin = 8;


const int detectionThreshold = 600;


const unsigned long buzzerDuration = 1000;
const unsigned long detectionCooldown = 3000;


bool detectionActive = false;
unsigned long lastDetectionTime = 0;


void setup() {
  Serial.begin(57600);


  pinMode(metalSensorPin, INPUT);
```

```
  pinMode(buzzerPin, OUTPUT);


  digitalWrite(buzzerPin, LOW);


  Serial.println("METAL_DETECTOR_READY");
}


void loop() {
  int sensorValue = analogRead(metalSensorPin);
  unsigned long currentTime = millis();


  if (sensorValue > detectionThreshold &&
      (currentTime - lastDetectionTime > detectionCooldown)) {
    detectionActive = true;
    lastDetectionTime = currentTime;


    digitalWrite(buzzerPin, HIGH);
    delay(buzzerDuration);
    digitalWrite(buzzerPin, LOW);


    Serial.print("METAL_DETECTED:");
    Serial.println(sensorValue);
  }
  else {
    Serial.print("CLEAR:");
    Serial.println(sensorValue);
  }
  delay(100);
}
```

APPENDIX B: HARDWARE AND ROS DATA FLOW SUMMARY

B.1 Hardware Interface Summary

Connection / Interface	Source	Destination	Purpose
USB / Serial	Raspberry Pi 4	Arduino Nano	ROS Arduino Bridge communication
Motor control signals	Arduino Nano	L298N motor driver	Direction and speed control
Motor output	L298N motor driver	76 RPM encoder motors	Rover wheel actuation
LiDAR USB	RPLidar A1	Raspberry Pi 4	LiDAR scan input for ROS
Analog detector output	DIY metal detector	Arduino UNO analog input	Metal detection voltage signal
Main power	3S 11.1 V LiPo battery	Motor driver / rover actuation system	Power for motors and actuation
Pi power	Power bank	Raspberry Pi 4	Stable power for onboard ROS computer

B.2 Important ROS2 Topics and Frames

ROS Topic / Frame	Purpose in the Rover System
`/cmd_vel`	Velocity command topic used to command rover motion
`/diff_cont/cmd_vel_unstamped`	Command input to the differential-drive controller
`/scan`	LiDAR scan topic used for mapping, costmaps, and navigation
`/odom`	Odometry information used for pose estimation
`/tf`	Dynamic transform data between coordinate frames
`/tf_static`	Static transform data for fixed robot frames
`map`	Global map frame used by SLAM and navigation
`odom`	Odometry frame used for local motion estimation
`base_link`	Main robot body frame
`base_footprint`	Ground-projected robot base frame
`laser_frame`	LiDAR sensor frame